

2.3.3 PHP预定义变量

+ PHP提供了大量的预定义变量，这些变量大多数依赖于服务器的版本及其配置。预定义变量可以在程序或文件的任何地方使用。

预定义变量名	作用
\$GLOBALS	包含指向当前程序中全局范围内有效的变量
\$SERVER	该全局变量是一个包含诸如头信息、路径和脚本位置的数组
\$_GET	通过HTTP的GET方法提交至脚本的表单变量。
\$_POST	通过HTTP的POST方法提交至脚本的表单变量。
\$_FILE	通过HTTP的POST文件上传提交至脚本的变量。
\$_COOKIE	通过HTTP的Cookies方法提交至脚本的变量。

2.4.2 PHP预定义常量

+ 在PHP中，除了用户可以自己定义常量，PHP中默认定义了一些常量，这些都是国际比较通用的。在编程过程中我们可以直接拿来使用而不需要定义。很多常量是由不同的扩展库定义的，只有在加载了这些扩展库时才会出现，这个我们在以后会在需要时候讲解。这里我们介绍一些常用的预定义常量，比如常量M_PI，它就表示数学中的 π 。

常量名	作用说明
__FILE__	返回当前文件的名称（注意下划线都是两个）
__LINE__	返回当前代码所在的行号（注意下划线都是两个）
__FUNCTION__	返回所在函数的函数名（注意下划线都是两个）
__CLASS__	返回所在类的类名（注意下划线都是两个）
PHP_OS	返回操作系统的名称
PHP_VERSION	返回当前PHP服务器的版本
TRUE	代表布尔值，真
FALSE	代表布尔值，假
NULL	代表空值
M_PI	数学中的 π

fun_ref.php :主要是包含对引用赋值的分析

```
<?php
```

/*通过这种方式\$a=test();得到的其实不是函数的引用返回，这跟普通的函数调用没有区别。

\$a=test()方式调用函数，只是将函数的值赋给\$a而已，

而\$a做任何改变都不会影响到函数中的\$b

而通过\$a=&test()方式调用函数呢，

他的作用是将 return \$b 中的\$b 变量的内存地址与\$a 变量的内存地址指向了同一个地方。

```

*/

// $b=0;
function &test()
{
    static /*global*/ $b; //申明一个静态变量
    //这里返回值 b 必须要
    $b=$b+1;
    echo $b;
    return $b;
}

// 在 PHP 中， 函数定义时没有使用 &，调用时使用了 &：不会报错，但 $a 仍然是 $b
// 的值的拷贝，而不是引用。
// 函数定义时没有使用 &，调用时也没有使用 &：$a 是 $b 的值的拷贝，而不是引用。
// 函数定义时使用了 &，调用时使用了 &：$a 是 $b 的引用，修改 $a 会同时修改 $b。
// 函数定义时使用了&，那么就一定得和静态变量配合使用，不然非静态变量执行完后就在内存
// 中消失了，此时就无法再返回对该变量的引用了。
// 函数定义时使用了 &，调用时没有使用 &：不会报错，但 $a 仍然是 $b 的值的拷贝，而
// 不是引用。
$a=test(); //这条语句会输出 $b 的值 为 1

$a=5;

$a=test(); //这条语句会输出 $b 的值 为 2

$a=&test(); //这条语句会输出 $b 的值 为 3

$a=5;

$a=test(); //这条语句会输出 $b 的值 为 6
?>

```

fun_var.php : 主要是给出了可变函数的使用方法

```

<?php
/*
PHP 支持可变函数的概念。这意味着如果一个变量名后有圆括号，
PHP 将寻找与变量的值同名的函数，并且尝试执行它。
*/
function foo() {
    echo "In foo()<br />\n";
}

function bar($arg = '') {
    echo "In bar(); argument was '$arg'.<br />\n";
}

```

```

}

$func = 'foo';

$func(); // 执行 foo(); 命令行中输出: In foo()<br />

$func = 'bar';

$func('test'); // 执行 bar(); 命令行中输出: In bar(); argument was
'test'.<br />
?>
<br>

<?php
/*可变函数的语法来调用一个对象的方法。*/
class Foo
{
    function Variable()
    {
        $name = 'Bar';
        $this->$name(); // This calls the Bar() method
    }

    function Bar()
    {
        echo "This is Bar";
    }
}

$foo = new Foo();
$funcname = "Variable";
$foo->$funcname(); // This calls $foo->Variable()

// 命令行执行输出: This is Bar
?>

```

Foreach.php

```

<?php
/*
foreach foreach 语句用于循环遍历数组。
每进行一次循环，当前数组元素的值就会被赋值给 value 变量
（数组指针会逐一地移动） - 以此类推。

```

```
*/
```

```
?>
```

```
<?php
```

```
$arr = array("one", "two", "three");
```

```
foreach ($arr as $value)
```

```
{
```

```
    //echo "Value: " . $value . "<br>";
```

```
    echo "Value: $value" . "<br>";
```

```
    //echo 'Value: $value' . "<br>";
```

```
}
```

```
?>
```

```
<?php
```

```
/*
```

从 PHP 5 开始，你可以通过在变量前面使用&操作符来修改数组的元素，这样的话分配的是引用，而不是值。

```
*/
```

```
$arr = array(1, 2, 3, 4);
```

```
foreach ($arr as &$amp;value) {
```

```
    print $value. "<br>";
```

```
    $value = $value * 2;
```

```
}
```

//reset(\$arr);unset() 用于销毁指定的变量，释放其占用的内存。销毁变量后，变量名将不再指向任何值。

```
unset($value);
```

```
foreach ($arr as $value) {
```

```
    print $value. "<br>";
```

```
}
```

```
// $arr is now array(2, 4, 6, 8)
```

```
unset($value); //break the reference with the last element
```

```
?>
```

```
<?php
```

```
$language=array("ASP", "PHP", "JSP");
```

//list(\$key,\$value) 和 each() 一起使用是将数组当前指针所指向单元的键/值对分别赋值给变量

```
while( (list($key,$value)=each($language)) ) {
```

```
    echo $key."=>".$value;
```

```
    echo "<br>";
```

```
}
```

```
?>
```

```
<?php
```

```
$arr = array("Apple", "Banana", "Orange");
```

//reset() 函数把数组的内部指针指向第一个元素，并返回这个元素的值。 若失败，则返回 FALSE。

```
reset($arr);
```

//each() 会将作用的数组的当前单元的键/值对返回，并且将数组指针向下移动一个位

```
$i=0;
```

```
while (list(, $value) = each($arr)) {
```

```
    echo "Fruit: $value<br />\n";
```

```
}
```

```
foreach ($arr as $value) {
```

```
    $i++;
```

```
    if ($i > 2) break;
```

```
    echo "Fruit: " . $value. "<br />\n";
```

```
    echo "Fruit: $value <br />\n";
```

```
}
```

```
?>
```

```
<?php
```

```
/*
```

在老版本的 PHP 中 list() 是和 each() 一起用来遍历数组的，

但是在现在流行 PHP5 中已经被 foreach(\$array as \$key=>\$value) 给代替，

所以 list() 可以说已经没有什么作用。

但是你试图将数组的前面几个元素的值赋给 list() 括号中所列的变量时还是有点用的

```
*/
```

```
?>
```

```
<?php
```

//Program List: 更多 foreach 的使用例子

```
$arr = array("Apple", "Banana", "Orange");
```

```
reset($arr);
```

```
while (list($key, $value) = each($arr)) {
```

```
    echo "Key: $key; Value: $value<br />\n";
```

```
}
```

```
reset($arr);
```

```
foreach ($arr as $key => $value) {
```

```
    echo "Key: $key; Value: $value<br />\n";
```

```
}
```

```
?>
```

```

<?php
/* foreach example 1: value only */

$a = array(1, 2, 3, 17);

foreach ($a as $v) {
    echo "Current value of \$a: $v.\n <br>";
}

/* foreach example 2: value (with its manual access notation printed
for illustration) */

$a = array(1, 2, 3, 17);

$i = 0; /* for illustrative purposes only */

foreach ($a as $v) {
    echo "\$a[$i] => $v.\n <br>";
    $i++;
}

/* foreach example 3: key and value */

$a = array(
    "one" => 1,
    "two" => 2,
    "three" => 3,
    "seventeen" => 17
);

foreach ($a as $k => $v) {
    echo "\$a[$k] => $v.\n";
}

/* foreach example 4: multi-dimensional arrays */
$a = array();
$a[0][0] = "a";
$a[0][1] = "b";
$a[1][0] = "y";
$a[1][1] = "z";

foreach ($a as $v1) {
    foreach ($v1 as $v2) {
        echo "$v2\n";
    }
}

```

```

    }
}

/* foreach example 5: dynamic arrays */

foreach (array(1, 2, 3, 4, 5) as $v) {
    echo "$v\n";
}
?>

```

```

Value: one
Value: two
Value: three
1
2
3
4
2
4
6
8
0=>ASP
1=>PHP
2=>JSP
Fruit: Apple
Fruit: Banana
Fruit: Orange
Fruit: Apple
Fruit: Apple
Fruit: Banana
Fruit: Banana
Key: 0; Value: Apple
Key: 1; Value: Banana
Key: 2; Value: Orange
Key: 0; Value: Apple
Key: 1; Value: Banana
Key: 2; Value: Orange
Current value of $a: 1.
Current value of $a: 2.
Current value of $a: 3.
Current value of $a: 17.
$a[0] => 1.
$a[1] => 2.
$a[2] => 3.
$a[3] => 17.
$a[one] => 1. $a[two] => 2. $a[three] => 3. $a[seventeen] => 17. a b y z 1 2 3 4 5

```

5.5 数组内部指针控制函数

- + 数组的内部指针是数组内部的组织机制，它可以指向数组中任意一个元素。数组内部指针默认是指向数组的第一个元素的。通过控制指针的移动，可以访问到数组中的任意一个元素。数组内部指针的控制，需要使用以下函数：
- + `current()`——返回当前元素的内容。`current()`并不移动指针。如果内部指针指向超出了元素列表的末端，`current()`返回FALSE。
- + `key()`——返回当前元素的索引值
- + `next()`——返回数组内部指针指向的下一个元素的内容，当没有更多元素时返回FALSE。需要注意的是如果数组包含空的元素，或者元素的值是0则本函数碰到这些元素也返回FALSE。因此`next()`不可用做循环的执行判断语句。`next()`和`current()`的行为类似，只有一点区别，在返回值之前将内部指针向前移动一位。这意味着它返回的是下一个数组单元的值并将数组指针向前移动了一位。
- + `prev()`——返回数组内部指针指向的前一个元素的值，或当没有更多元素时返回FALSE。需要注意的是如果数组包含空的元素，或者元素的值是0则本函数碰到这些元素也返回FALSE。
- + `end()`——将数组的内部指针移动到最后一个元素并返回其值。
- + `reset()`——将数组的内部指针倒回到第一个元素并返回第一个数组元素的值，如果数组为空则返回FALSE。

5.6 PHP中的预定义数组

- + 在学习函数的时候，我们在学习了自定义函数后，还学习了PHP的系统函数。我们前面所学的数组的内容，就是自定义的数组，PHP也提供了一些已经由系统定义好的数组。这些数组在全局范围内自动生效。他们包含了来自web服务器、客户端、运行环境和用户输入等数据。表列出了PHP常用的预定义数组及其说明。

预定义数组名	说明
\$GLOBALS	包含了全部变量的全局组合数组。变量的名字就是数组的键。
\$_SERVER	包含了诸如头信息、路径、以及脚本位置等信息的数组。
\$_GET	通过URL参数传递给当前脚本的变量的数组。
\$_POST	通过 HTTP POST 方法传递给当前脚本的变量的数组。
\$_REQUEST	默认情况下包含了 \$_GET、\$_POST和\$_COOKIE 的数组。
\$_FILES	通过 HTTP POST 方式上传到当前脚本的项目的数组。
\$_SESSION	当前脚本可用 SESSION 变量的数组。
\$_COOKIE	通过 HTTP Cookies 方式传递给当前脚本的变量的数组。
\$_ENV	通过环境方式传递给当前脚本的变量的数组。

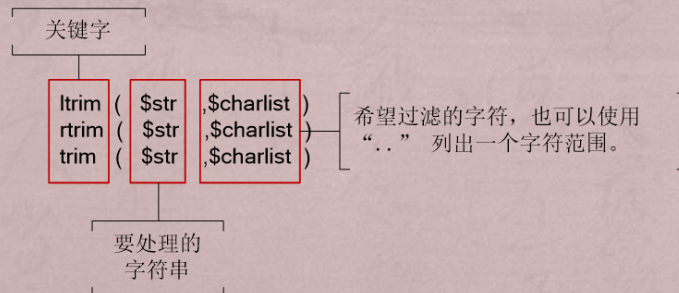
8.1 常用的字符串输出函数

- + 在前面章节的学习中，我们常常使用的输出字符串的函数就是echo()，在PHP中还有一些其他不同功能的字符串输出函数。我们先用表列出它们，然后再分别讲解它们。

函数名	描述
echo() (无返回值)	输出字符串
print() (有返回值)	输出一个或者多个字符串
die()	输出字符串并退出当前脚本 (相当于可以设置断点，可以用来调试程序)
printf()	输出格式化字符串
sprintf()	把格式化的字符串写入一个变量中

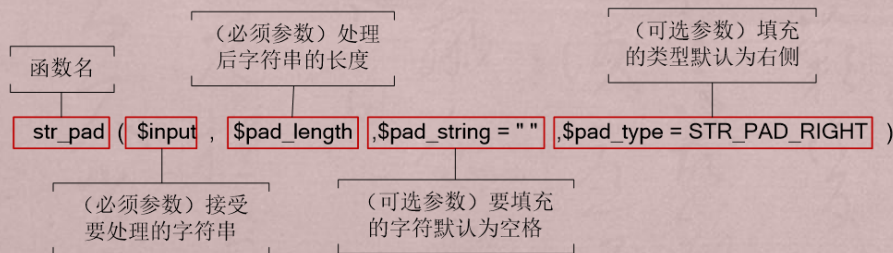
8.2.1 删除和填补字符函数

- + 空格也是一个有效的字符，在字符串中也会占据一个位置。用户在输入数据时常会在无意间输入都多余的空格，这在像登录系统时候常会出现错误。在PHP接收到用户传来的数据后常会删除空格或者其他没有意义的字符串。我们通常使用ltrim(), rtrim()和trim()来完成这些工作。它们的语法如图所示。
- + ltrim()用于去除字符串开始处的空白字符或者其他字符。rtrim()用于去除字符串结尾处的空白字符或者其他字符。trim()用于去除字符串开头和结尾处的空白字符或者其他字符（三者都不能去除中间的特殊字符）。这三个函数都必须传入第一个参数，如果不指定第二个参数，它们将去除这些字符：
- + " " (ASCII 32 (0x20)), 普通空格符。
- + "\t" (ASCII 9 (0x09)), 制表符。
- + "\n" (ASCII 10 (0x0A)), 换行符。
- + "\r" (ASCII 13 (0x0D)), 回车符。
- + "\0" (ASCII 0 (0x00)), 空字节符。
- + "\x0B" (ASCII 11 (0x0B)), 垂直制表符。



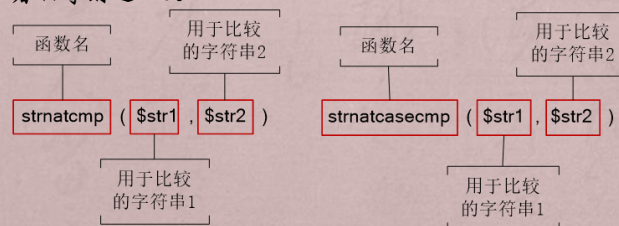
8.2.1 删除和填补字符函数

- + 这一组处理字符串的函数还是比较简单的，我们就只通过几个简单的示例讲解即可。上面我们讲解的是删除字符串的函数，下面我们就来讲解一个填充字符串的函数str_pad(), 它的语法如图所示。
- + 在图中：
- + 如果pad_length的值是负数，小于或者等于输入字符串的长度，不会发生任何填充。
- + 如果填充字符串的长度不能被pad_string整除，那么pad_string可能会被缩短。
- + 可选的pad_type参数的可能值为STR_PAD_RIGHT, STR_PAD_LEFT或STR_PAD_BOTH。如果没有指定pad_type，默认是STR_PAD_RIGHT。
- + (1)演示str_pad()的使用及其使用后的输出结果。



8.3.2 STRNATCMP()和STRNATCASECMP()函数

- + PHP除了提供了使用字节ASCII值进行比较的函数外，还提供了比较接近“自然排序”的字符串比较方法。就比如使用字节ASCII码值比较时，“9”是大于“33”的，因为“9”的ASCII值大于“33”第一个字符的值。使用自然排序法则“33是大于“9”的。PHP中可以使用strnatcmp()和strnatcasecmp()来按照自然排序的方式比较两个字符串。它们的语法如图所示。
- + strnatcmp()和strnatcasecmp()语法的区别是strnatcasecmp()不区分比较的字符串的大小写。它们会把字符串中的数字部分按照数字大小进行比较，字母部分按照“A<B<C<...Z<a<b<c<...z”的方式进行比较。比较结束后它们均会返回一个整型数：
- + 如果str1小于str2，返回负数。
- + 如果str1大于str2，返回正数。
- + 二者相等则返回0。



Charhtml.php

```
<!DOCTYPE html>
<html>
<body>
```

```
<?php
```

```
/*
```

htmlspecialchars() 函数把预定义的字符转换为 HTML 实体。

预定义的字符是：

& （和号）成为 &

" （双引号）成为 "

' （单引号）成为 '

< （小于）成为 <

> （大于）成为 >

```
*/
```

```
$str = "This is some <b>bold</b> text.";
```

```
echo htmlspecialchars($str);
```

```
?>
```

```
<?php
```

//nl2br() 函数在字符串中的每个新行 (\n) 之前插入 HTML 换行符 (
 或
)。

```
echo nl2br("One line.\nAnother line.");
```

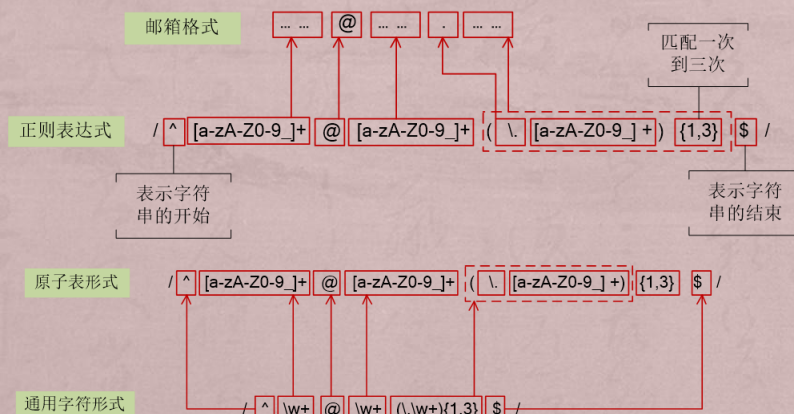
```
?>
```

```
</body>
```

```
</html>
```

1.通用字符类型

- + 知道了邮箱格式以后我们就可以使用正则表达式来表示了，如图所示。
- + 由于原子表 "[a-zA-Z0-9]" 表示匹配所有字母、数字和下划线，而通用字符类型 "\w" 也是表示相同的功能。因此我们可以把图中的原子表替换为通用字符类型，如图所示。
- + 我们可以看到通用字符形式的正则表达式在很大程度上精简了长度，也使得表达式更加容易阅读。因此我们在允许的情况下应该多使用通用字符形式。



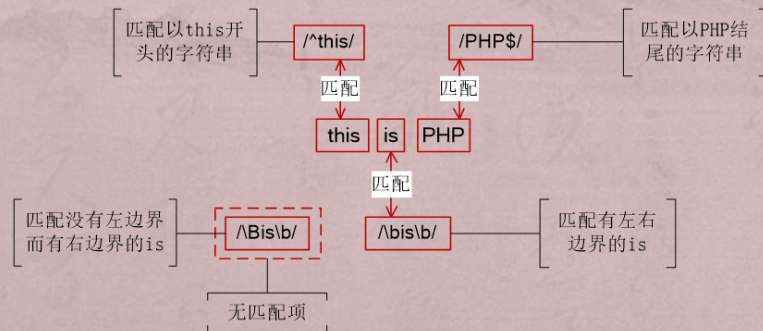
8.4.3 元字符

- + 元字符就是用于构建正则表达式的具有特殊含义的字符。如果要在正则表达式中包含元字符本身，则必须在其前面加上“\”进行转义。正则表达式有以下特殊字符，如表所示。

字符	描述
\$	匹配输入字符串的结尾位置。如果设置了 RegExp 对象的 Multiline 属性，则 \$ 也匹配 '\n' 或 '\r'。
()	标记一个子表达式的开始和结束位置。子表达式可以获取供以后使用。
*	匹配前面的子表达式零次或多次。
+	匹配前面的子表达式一次或多次。
.	匹配除换行符 \n 之外的任何单字符。
[]	匹配方括号中指定的任意一个原子。
[^]	匹配除方括号内原子以外的任意字符。
?	匹配前面的子表达式零次或一次，或指明一个非贪婪限定符。
\	将下一个字符标记为特殊字符、或原义字符、或向后引用、或 \n 转义符。
^	匹配输入字符串的开始位置，除非在方括号表达式中使用，此时它表示不接受该字符集合。
	指明两项之间的一个选择。
\s	匹配任何空白字符，包括空格、制表符、换行符等。
\S	匹配任何非空白字符。
\w	匹配匹配任意一个数字、字母或下划线。
\W	匹配除数字、字母或下划线的任意字符。
\b	匹配单词的边界。
\B	匹配除单词边界以外的部分。
{n}	表示其前面的原子正好出现 n 次。
{n,}	表示其前面的原子至少出现 n 次。
{n,m}	表示其前面的原子至少出现 n 次，至多出现 m 次。

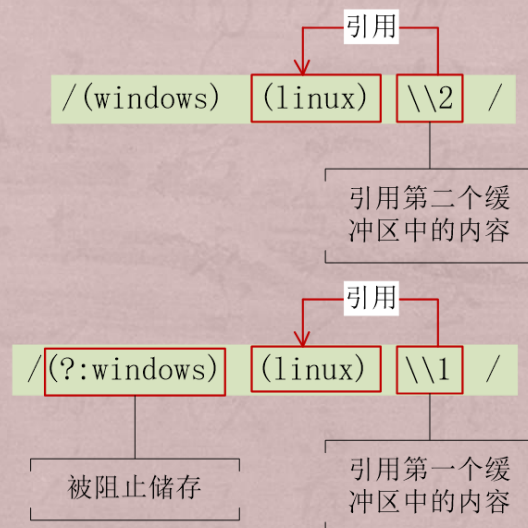
3.边界限制

- + 边界限制是用来限制字符串或者单词的边界，以获得更准确的匹配结果。“^”和“\$”分别表示字符串的开始和结束。“\b”用于匹配字符串中每个单词的前或后边界。“\B”表示非单词边界比如这有一段字符串“this is PHP”，我们来尝试使用不同的正则表达式来匹配它们，如图所示。



8.后向引用

- + 当然这种更新缓冲区是可以由我们控制的，我们使用“?:”来阻止模式单元被储存，如图所示。



10. 模式修正符

- + 模式修正符在正则表达式的定界符之外使用。例如“/php/i”，其中的“i”就是模式修正符，用来在匹配时不区分大小写。模式修正符也是可以组合使用的。例如“/php/ism”就是“i”、“s”、“m”三个修正符组合使用的。模式修正符对编写简洁的正则表达式有很大的帮助。表列出了一些修正符以及功能描述。

模式修正符	描述
i	不区分大小写的匹配
m	将字符串视为多行,不管是哪行都能匹配
s	将字符串视为单行,换行符作为普通字符;
x	将模式中的空白忽略
e	配合函数preg_replace()使用,可以把匹配来的字符串当作正则表达式执行
A	强制从目标字符串开头匹配
D	如果使用\$限制结尾字符,则不允许结尾有换行
S	如果设定了此修正符则会进行额外的分析
U	只匹配最近的一个字符串;不重复匹配

9.2.1 进制间的转换

- + 我们知道计算机使用的是进制数，有的时候我们需要把一个数转换为其他进制的数，这就需要用到转换进制的函数，常用的进制间转换的函数及功能描述如表所示。

函数名	功能描述
decbin()	将十进制数转换为二进制数
decoct()	将十进制数转换为八进制数
dechex()	将十进制数转换为十六进制数
bindec()	将二进制数转换为十进制数
hexdec()	将十六进制数转换为十进制数
base_convert()	任意进制间数字的转换

函数名	功能描述
abs()	取得数值的绝对值
ceil()	进一法取整
floor()	舍去法取整
fmod()	返回除法的浮点数余数
round()	对浮点数四舍五入法取整

函数名	功能描述
abs()	求绝对值
sqrt()	求平方根
pow()	指数表达式
fmod()	返回除法的浮点余数
log()	自然对数

函数名	功能描述
is_nan()	判断参数是否为一个非数值
is_finite()	判断参数是否为一个有限值
is_infinite()	判断参数是否为一个无限值

"r"	只读方式打开，将文件指针指向文件头
"r+"	读写方式打开，将文件指针指向文件头
"w"	写入方式打开，将文件指针指向文件头并将文件大小截为零。如果文件不存在则尝试创建
"w+"	读写方式打开，将文件指针指向文件头并将文件大小截为零。如果文件不存在则尝试创建
"a"	写入方式打开，将文件指针指向文件末尾。如果文件不存在则尝试创建
"a+"	读写方式打开，将文件指针指向文件末尾。如果文件不存在则尝试创建
"x"	创建并以写入方式打开，将文件指针指向文件头。如果文件已存在，则fopen()调用失败并返回FALSE，并生成一条E_WARNING级别的错误信息。如果文件不存在则尝试创建
"x+"	创建并以读写方式打开，将文件指针指向文件头。如果文件已存在，则fopen()调用失败并返回FALSE，并生成一条E_WARNING级别的错误信息。如果文件不存在则尝试创建

文件上传

10.4.2 认识预定义变量\$_FILES

- + \$_FILES变量储存着上传文件的相关信息，对于上传文件是很重要的。它是一个二维数组，它的元素名及含义如表所示。
- + (1)演示\$_FILES结合HTML表单取得上传文件信息。

数组元素	保存的信息
\$_FILES[filename][name]	保存上传文件的文件名
\$_FILES[filename][size]	保存上传文件的大小
\$_FILES[filename][tmp_name]	保存上传文件的临时名称
\$_FILES[filename][type]	保存上传文件的类型
\$_FILES[filename][error]	保存上传文件结果的代号，0则表示成功

fileupload.html:上传文件

```
<body>
<table id="divContainer" style="HEIGHT: 100%; WIDTH: 380" border="0">
  <!--
  <form> 标签的 enctype 属性规定了在提交表单时要使用哪种内容类型。
  在表单需要二进制数据时，比如文件内容，请使用 "multipart/form-data"。

  <input> 标签的 type="file" 属性规定了应该把输入作为文件来处理。
  举例来说，当在浏览器中预览时，会看到输入框旁边有一个浏览按钮。
  -->

  <form method="post" name="frmUpload" enctype="multipart/form-data"
  action="upload_file.php">

    <tr height="35"><td align="right" valign="bottom">多文件上传
  </td></tr>
    <tr><td align="center" valign="top">
      <table class="Transex" border="1" cellpadding="0" cellspacing="0" >
        <tr>
          <td nowrap><label class="FieldLabel"> 文件 1</label></td>
```



```

$spath=AddSlashes(dirname(__FILE__)) . "\\upload\\";
//echo AddSlashes(dirname(__FILE__)); die();
echo $spath;//die();
for($i=1;$i<=2;$i++) {
    $files="afile$i";
    echo $files. " Kb<br />";
//用户只能上传 pdf 文件，文件大小必须大于 20 kb:
//if (($FILES[$files]["type"] == "application/pdf")
//&& ($FILES[$files]["size"] > 20000))
//{
    if ($FILES[$files]["error"] > 0)
    {
        echo "Error: " . $FILES[$files]["error"] . "<br />";
    }
    else
    {
        echo "Upload: " . $FILES[$files]["name"] . "<br />";
        echo "Type: " . $FILES[$files]["type"] . "<br />";
        echo "Size: " . ($FILES[$files]["size"] / 1024) . " Kb<br
/>";
        echo "Stored in: " . $FILES[$files]["tmp_name"] . " Kb<br
/>";
    }
}

```

/*

通过使用 PHP 的全局数组 \$_FILES，你可以从客户计算机向远程服务器上传文件。

第一个参数是表单的 input name，

第二个下标可以是 "name", "type", "size", "tmp_name" 或 "error"。就像这样：

```

$_FILES["file"]["name"] - 被上传文件的名称
$_FILES["file"]["type"] - 被上传文件的类型
$_FILES["file"]["size"] - 被上传文件的大小，以字节计
$_FILES["file"]["tmp_name"] - 存储在服务器的文件的临时副本的名称
$_FILES["file"]["error"] - 由文件上传导致的错误代码
*/

```

```

echo $_FILES[$files]['tmp_name'];
//is_uploaded_file() 函数判断指定的文件是否是通过 HTTP POST 上传的
if (is_uploaded_file($_FILES[$files]['tmp_name'])) {
    $filename = $_FILES[$files]['name'];
    $localfile = $spath . $filename;
    echo $localfile;
//move_uploaded_file() 函数将上传的文件移动到新位置。
move_uploaded_file($_FILES[$files]['tmp_name'], $localfile);

```

```

    }
}
// }
}
echo "<br><b>You have uploaded files successfully</b><br>";
exit;

}
?>

```

运行结果:

多文件上传

文件1	选择文件	0.jpg
文件2	选择文件	1.jpg
		确定 取消

```

E:\xampp\www\upload\afile1 Kb
Upload: 0.jpg
Type: image/jpeg
Size: 508.1953125 Kb
Stored in: E:\xampp\tmp\php6A5A.tmp Kb
E:\xampp\tmp\php6A5A.tmpE:\xampp\www\upload\0.jpgfile2 Kb
Upload: 1.jpg
Type: image/jpeg
Size: 508.771484375 Kb
Stored in: E:\xampp\tmp\php6A5B.tmp Kb
E:\xampp\tmp\php6A5B.tmpE:\xampp\www\upload\1.jpg
You have uploaded files successfully

```

fileuploadcompressed.html: 图片压缩存储

```

<body>
<table id="divContainer" style="HEIGHT: 100%; WIDTH: 380" border="0">
  <!--
  <form> 标签的 enctype 属性规定了在提交表单时要使用哪种内容类型。
  在表单需要二进制数据时，比如文件内容，请使用 "multipart/form-data"。

```

```

<input> 标签的 type="file" 属性规定了应该把输入作为文件来处理。
举例来说，当在浏览器中预览时，会看到输入框旁边有一个浏览按钮。

```



```

if($_POST["ifupload"]=="1") {
/*
addslashes() 函数在指定的预定义字符前添加反斜杠。
这些预定义字符是：
单引号 (')
双引号 (")
反斜杠 (\)
NULL
*/
//在程序中，有时我们会看到这样的路径写法，"D:\\Driver\\Lan" 也就是两个反斜杠来
分隔路径。
$path=AddSlashes(dirname(__FILE__)) . "\\upload\\";

echo $path;
for($i=1;$i<=2;$i++) {
    $files="afile$i";
    echo $files. " Kb<br />";
//用户只能上传 pdf 文件，文件大小必须大于 20 kb:
//if (($_FILES[$files]["type"] == "application/pdf")
//&& ($_FILES[$files]["size"] > 20000))
//{
    if ($_FILES[$files]["error"] > 0)
    {
        echo "Error: " . $_FILES[$files]["error"] . "<br />";
    }
    else
    {
        echo "Upload: " . $_FILES[$files]["name"] . "<br />";
        echo "Type: " . $_FILES[$files]["type"] . "<br />";
        echo "Size: " . ($_FILES[$files]["size"] / 1024) . " Kb<br
/>";
        echo "Stored in: " . $_FILES[$files]["tmp_name"] . " Kb<br
/>";
    }

/*

```

通过使用 PHP 的全局数组 \$_FILES，你可以从客户计算机向远程服务器上传文件。

第一个参数是表单的 input name，

第二个下标可以是 "name", "type", "size", "tmp_name" 或 "error"。就像这样：

\$_FILES["file"]["name"] - 被上传文件的名称

```

_FILES["file"]["type"] - 被上传文件的类型
_FILES["file"]["size"] - 被上传文件的大小，以字节计
_FILES["file"]["tmp_name"] - 存储在服务器的文件的临时副本的名称
_FILES["file"]["error"] - 由文件上传导致的错误代码
*/
echo $_FILES[$files]['tmp_name'];

//is_uploaded_file() 函数判断指定的文件是否是通过 HTTP POST 上传的
if (is_uploaded_file($_FILES[$files]['tmp_name'])) {
    $filename = $_FILES[$files]['name'];
    $src_type = $_FILES[$files]['type'];
    $src_type = explode('/', $src_type)[count($src_type)];
    // 从 MIME 类型中提取文件扩展名。MIME 类型通常由两部分组成：类型/子类型。
    // 例如：
    // text/html：表示 HTML 文档。
    // image/jpeg：表示 JPEG 图像。
    // application/json：表示 JSON 格式的数据。

    $localfile = $path . $filename;
    $localfile_small = $path . "s" . $filename; //小图路径
    echo $localfile;

    $src_fnc = 'imagecreatefrom' . $src_type ; //创建图像资源函数 对
应函数 imagecreatefrompng 或者 imagecreatefromjpeg
    $src_image = $src_fnc($_FILES[$files]['tmp_name']); //临时图
片资源

    $src_w = imagesx($src_image); //上传图片的宽
    $src_h = imagesy($src_image); //高
    if ($src_w > 600)
    {
        $dst_w = 300; //固定 300,
        $dst_h = floor((300.0/$src_w)*$src_h); //同样的比例缩放
    }
    else
    {
        $dst_w = floor($src_w/3); //这里 目标图片的宽是源图片的三分之
一,
        $dst_h = floor($src_h/3); //同样的
    }

    $dst_image = imagecreatetruecolor($dst_w, $dst_h); //此时的画
布为原来图片的 1/3 或比例分之一
    imagecopyresampled($dst_image, $src_image, 0, 0, 0, 0,
$dst_w, $dst_h, $src_w, $src_h); //将源图像按比例缩放到目标图像资源中。

    $out_fnc = 'image' . $src_type; //imagepng imagejpeg

```

```

        if( !($out_fnc($dst_image , $localfilesmall) &&
file_exists($localfilesmall)){
            //缩略图保存成功了，文件路径就是$path;
            // $out_fnc($dst_image, $localfilesmall);: 将缩略图保存到
指定路径。
            // file_exists($localfilesmall): 检查文件是否成功保存。
        } else{
            //保存失败了，检查错误
        }
    }

    //move_uploaded_file() 函数将上传的文件移动到新位置。

    move_uploaded_file($_FILES[$files]['tmp_name'], $localfile);
}

// }
}

echo "<br><b>You have uploaded files successfully</b><br>";
exit;

}
?>

```

多文件上传

文件1	选择文件	0.jpg
文件2	选择文件	1.jpg
		确定 取消

```

E:\xampp\www\upload\afile1 Kb
Upload: 0.jpg
Type: image/jpeg
Size: 508.1953125 Kb
Stored in: E:\xampp\tmp\php476C.tmp Kb
E:\xampp\tmp\php476C.tmpE:\xampp\www\upload\0.jpgafile2 Kb
Upload: 1.jpg
Type: image/jpeg
Size: 508.771484375 Kb
Stored in: E:\xampp\tmp\php477D.tmp Kb
E:\xampp\tmp\php477D.tmpE:\xampp\www\upload\1.jpg
You have uploaded files successfully

```


10.4.1 配置PHP.INI文件

- + 在上传文件之前首先要配置php.ini中的如下选项，如图所示。
- + 由于我们使用的是集成环境，这些选项都是已经配置好的，当然读者再确认一次最好。这些选项的含义如下：
- + memory_limit: PHP中给一个指令分配的内存空间，以MB为单位。
- + max_execution_time: PHP中执行一条指令可以使用的最长时间。
- + file_uploads: 是否支持文件上传。
- + upload_tmp_dir: 上传文件的临时目录。
- + upload_max_filesize: 允许上传文件的最大值，以MB为单位。
- + 在配置好这些选项后就为文件上传做好了基础准备。

```
460 memory_limit = 128M
442 max_execution_time = 30
877 file_uploads = On
878
879 ; Temporary directory for HTTP uploads
880 ; (should be specified).
881 ; http://php.net/upload-tmp-dir
882 upload_tmp_dir = "D:\xampp\tmp"
883
884 ; Maximum allowed size for uploaded files
885 ; http://php.net/upload-max-filesize
886 upload_max_filesize = 128M
```

MYSQL 数据库基础

2.NULL类型

- + 在PHP的类型中我也常常使用NULL。
MySQL中的“NULL”表示“没有值”或者“未知值”。在数据表中可以插入“NULL”作为值。也可以用来检测一个值是否为空值。如果对“NULL”进行数学运算，那么结果还是为“NULL”（用后面标红的性质就可以检测数据是否为空）。

3.类型转换

+ 与PHP类似的，在MySQL的表达式中，如果某个数值的类型与规定的类型不符，那么MySQL将会对将要操作的数值进行自动转换类型。例如：

+ $1+'3'$ // 会转换成 $1+3=4$

+ $1+'abc'$ // 会转换成 $1+0=1$ ，因为‘abc’不能转换为任何值，因此默认转换为0（区别于javascript中 $1+'3' = '13'$ ）

修改表结构

常见语法：添加列/删除列/修改列数据类型/修改列名称/添加主键/删除主键

ALTER TABLE 表名 ADD COLUMN 列名 数据类型 [约束条件];

ALTER TABLE 表名 DROP COLUMN 列名;

ALTER TABLE 表名 MODIFY COLUMN 列名 新数据类型 [约束条件];

ALTER TABLE 表名 CHANGE COLUMN 旧列名 新列名 新数据类型 [约束条件];

ALTER TABLE 表名 ADD PRIMARY KEY (列名);

ALTER TABLE 表名 DROP PRIMARY KEY;

前提是表中的数据都被删除后，才可以用alter修改表中的内容

12.4.1 数据库设计的第一范式

- + 最基本的范式
- + 如果数据库表中的所有字段值都是不可分解的原子值，就说明该数据库表满足了第一范式。数据库表中的字段都是单一属性的，不可再分。这个单一属性由基本类型构成，包括整型、实数、字符型、逻辑型、日期型等。
- + 第一范式的合理遵循需要根据系统的实际需求来定

12.4.2 数据库设计的第二范式

- + 数据库表中不存在非关键字段对任一候选关键字段的部分函数依赖（部分函数依赖指的是存在组合关键字中的某些字段决定非关键字段的情况），也即所有非关键字段都完全依赖于任意一组候选关键字。
- + 假定选课关系表为SelectCourse(学号, 姓名, 年龄, 课程名称, 成绩, 学分)，关键字为组合关键字(学号, 课程名称)，因为存在如下决定关系：

12.4.2 数据库设计的第二范式

- + (学号, 课程名称) $\rightarrow$ (姓名, 年龄, 成绩, 学分)
- 这个数据库表不满足第二范式，因为存在如下决定关系：
- (课程名称) $\rightarrow$ (学分)
- (学号) $\rightarrow$ (姓名, 年龄)
- 即存在组合关键字中的字段决定非关键字的情况。

12.4.2 数据库设计的第二范式

+ 由于不符合2NF，这个选课关系表会存在如下问题：

(1) 数据冗余：

同一门课程由 n 个学生选修，“学分”就重复 $n-1$ 次；同一个学生选修了 m 门课程，姓名和年龄就重复了 $m-1$ 次。

(2) 更新异常（即去更新的时候太麻烦了）：

若调整了某门课程的学分，数据表中所有行的“学分”值都要更新，否则会出现同一门课程学分不同的情况。

(3) 插入异常：

假设要开设一门新的课程，暂时还没有人选修。这样，即使有“学号”关键字，课程名称和学分也无法记录入数据库。

(4) 删除异常：

假设一批学生已经完成课程的选修，这些选修记录就应该从数据库表中删除。但是，与此同时，课程名称和学分信息也被删除了。很显然，这也会导致插入异常。

12.4.2 数据库设计的第二范式

+ 把选课关系表SelectCourse改为如下三个表：

学生：Student(学号, 姓名, 年龄)；

课程：Course(课程名称, 学分)；

选课关系：SelectCourse(学号, 课程名称, 成绩)。

+ 这样的数据库表是符合第二范式的，消除了数据冗余、更新异常、插入异常和删除异常。

+ 另外，所有单关键字的数据库表都符合第二范式，因为不可能存在组合关键字。

12.4.3 数据库设计的第三范式

- + 在第二范式的基础上，数据表中如果不存在非关键字段对任一候选关键字段的传递函数依赖则符合第三范式。所谓传递函数依赖，指的是如果存在" $A \rightarrow B \rightarrow C$ "的决定关系，则C传递函数依赖于A。
- + 因此，满足第三范式的数据库表应该不存在如下依赖关系：

关键字段 $\rightarrow$ 非关键字段x $\rightarrow$ 非关键字段y

12.4.3 数据库设计的第三范式

- + 假定学生关系表为Student(学号, 姓名, 年龄, 所在学院, 学院地点, 学院电话)，关键字为单一关键字"学号"，因为存在如下决定关系：

(学号) $\rightarrow$ (姓名, 年龄, 所在学院, 学院地点, 学院电话)

这个数据库是符合2NF的，但是不符合3NF，因为存在如下决定关系：

(学号) $\rightarrow$ (所在学院) $\rightarrow$ (学院地点, 学院电话)

12.4.3 数据库设计的第三范式

- + 即存在非关键字段"学院地点"、"学院电话"对关键字段"学号"的传递函数依赖。

它也会存在数据冗余、更新异常、插入异常和删除异常的情况，读者可自行分析得知。

把学生关系表分为如下两个表：

学生：(学号, 姓名, 年龄, 所在学院)；

学院：(学院, 地点, 电话)。

这样的数据库表是符合第三范式的，消除了数据冗余、更新异常、插入异常和删除异常。

注意看这个例子在把对应字段选出来时**列名的变化**

```
MariaDB [self_test]> select age*3 from  
testtable;
```

```
+-----+  
| age*3 |
```

```
+-----+
```

```
| 60 |
```

```
| 60 |
```

```
| 60 |
```

```
+-----+
```

```
3 rows in set (0.00 sec)
```

```
MariaDB [self_test]> select age*3 as age的三  
倍 from testtable;
```

```
+-----+  
| age的三倍 |
```

```
+-----+
```

```
| 60 |
```

```
| 60 |
```

```
| 60 |
```

```
+-----+
```

```
3 rows in set (0.00 sec)
```

字符操作符(LIKE和%)

如果你想查找不十分精确的数据, like 很好用:

```
♦ select * from authors where  
au_lname like 'St%'
```

上面的操作选出au_lname的第一个字母是s的记录,
试试小写

```
♦ select * from authors where  
au_lname like 'st%'
```

大小写没关系。

%是通配符, 代表多个字符(可以代表多个字符、一个
字符、空字符)。%也可以多个使用, 见下面例子。

下划线是单个字符通配符:

```
♦ select * from authors where zip  
like '946_9'
```

操作符(IN)

in可以简化你已经学过的一些查询, 或者说你不会用in
也没关系, 用前面学过的知识可以满足你的要求。我们看前
面的一个例子:

```
♦ select * from discounts where  
discount=5 or discount=6.7
```

我们用in来实现。

```
♦ select * from discounts where  
discount in (5,6.7) (不代表连续区间, 而是  
代表离散值)
```

不仅语句更短了, 而且更容易阅读, in也可用于字符类
型的字段。

Count()函数的使用:

count函数返回符合select语句中的查询条件的行数。

♦ **select count(*) from discounts
where discount>=5 and discount<10**

MariaDB [self_test]> select count(*) from testtable;

```
+-----+  
| count(*) |  
+-----+  
|      6 |  
+-----+
```

聚集函数(SUM)

sum函数返回一列中所有值的和。只能是一列，否则会报错。

♦ **select sum(discount) from
discounts where discount>=5 and
discount<10**

只对数值类型字段有效。

MariaDB [self_test]> select sum(*) from
testtable;
ERROR 1064 (42000): You have an error in your
SQL syntax; check the manual that corresponds
to your MariaDB server version for the right
syntax to use near '*) from testtable' at line 1

报错示例

MariaDB [self_test]> select sum(age) from
testtable;

```
+-----+  
| sum(age) |  
+-----+  
|    174 |  
+-----+
```

1 row in set (0.01 sec)

聚集函数(AVG)

avg函数计算一列的平均值。

♦ **select avg(discount) from
discounts where discount>=5 and
discount<10**

只对数值类型字段有效。

聚集函数(MIN) (MAX)

min函数找到一列的最小值。

♦ `select min(discount) from discounts where discount>=5 and discount<10`

可用于字符型字段。

max函数和min函数可以一块儿使用，查出值所在的范围。

♦ `select min(discount), max(discount) from discounts`

聚集函数(VAR)

var函数计算方差。

♦ `select var(discount) from discounts`

只对数值类型字段有效。

聚集函数(STDEV)

stdev函数计算标准差。

♦ `select stdev(discount) from discounts`

只对数值类型字段有效。

SELECT语句的子句

◆ 从数据库中检索行，并允许从一个或多个表中选择一个或多个行或列。虽然 SELECT 语句的完整语法较复杂，但是其主要的子句可归纳如下：

```
◆ SELECT select_list
[ INTO new_table ]
FROM table_source
[ WHERE search_condition ]
[ GROUP BY group_by_expression ]
[ HAVING search_condition ]
[ ORDER BY order_expression [ ASC | DESC ] ]
```

where、group by等是按顺序逐次执行选出来对应的表项
然后最后再select出相应要select的列出来

```
MariaDB [self_test]> select * from testtable;
```

```
+-----+-----+
| name | id | age |
+-----+-----+
| ??? | 1 | 20 |
| 李易阳 | 10 | 6 |
| ??? | 2 | 20 |
| 李易阳 | 3 | 20 |
| 011 | 4 | 12 |
| 012 | 5 | 14 |
| 013 | 6 | 88 |
+-----+-----+
```

```
MariaDB [self_test]> select id,avg(age),name
from testtable group by name ;
```

```
+-----+-----+
| id | avg(age) | name |
+-----+-----+
| 4 | 12.0000 | 011 |
| 5 | 14.0000 | 012 |
| 6 | 88.0000 | 013 |
| 1 | 20.0000 | ??? |
| 10 | 13.0000 | 李易阳 |
+-----+-----+
```

```
MariaDB [self_test]> select id,avg(age),name
from testtable group by name order by id;
```

```
+-----+-----+
| id | avg(age) | name |
+-----+-----+
| 1 | 20.0000 | ??? |
| 10 | 13.0000 | 李易阳 |
| 4 | 12.0000 | 011 |
| 5 | 14.0000 | 012 |
| 6 | 88.0000 | 013 |
+-----+-----+
```

第一个查询：select id, avg(age) from testtable
group by name order by id;

•合理部分：avg(age) 是聚合函数处理的结果，按 name 分组计算每个名字的平均年龄，这部分逻辑正确（每个 name 分组的平均年龄是确定的）。

•问题部分：SELECT 中包含 id，但 id 既不在 GROUP BY 中（分组依据是 name），也未

被聚合函数处理。
这意味着：每个 name 分组可能包含多条记录（对应多个 id），但结果中只显示一个 id，这个 id 是数据库从该分组内“随机”挑选的，不代表该分组的真实逻辑关联（如最小/最大/第一条 id），因此 id 列的值是不可靠的。

HAVING子句

having子句给用在group by子句中的数据加限制条件。
HAVING 通常与 GROUP BY 子句一起使用。如果不使用
GROUP BY 子句, HAVING 的行为与 WHERE 子句一样。

♦ SELECT au_lname, sum(price) 分组和,
count(au_lname) 计数 from titleview
group by au_lname having
sum(price)>20 order by sum(price)

HAVING可以让你在比较表达式中使用聚集函数, 而
WHERE则不行。

CROSS JOIN连接两个表

```
MariaDB [self_test]> select * from testtable ;
```

name	id	age	pmname
张三	1	15	篮球
李四	2	15	排球

```
MariaDB [self_test]> select * from sportstable;
```

index	name
1	篮球
2	排球

```
MariaDB [self_test]> select * from testtable  
cross join sportstable;
```

name	id	age	pmname	index	name
张三	1	15	篮球	1	篮球
李四	2	15	排球	1	篮球
张三	1	15	篮球	2	排球
李四	2	15	排球	2	排球

```
MariaDB [self_test]> select * from testtable ,  
sportstable;
```

name	id	age	pmname	index	name
张三	1	15	篮球	1	篮球
李四	2	15	排球	1	篮球
张三	1	15	篮球	2	排球
李四	2	15	排球	2	排球

容易看出下面两个select语句的效果是一样的

```

MariaDB [self_test]> select * from testtable;
+-----+-----+-----+-----+
| name | id | age | pmname |
+-----+-----+-----+
| 张三 | 1 | 15 | 篮球 |
| 李四 | 2 | 15 | 排球 |
+-----+-----+-----+

MariaDB [self_test]> select * from sportstable;
+-----+-----+
| index | name |
+-----+-----+
| 1 | 篮球 |
| 2 | 排球 |
+-----+-----+

```

```

MariaDB [self_test]> select * from testtable inner join sportstable on
testtable.pmname=sportstable.name;
+-----+-----+-----+-----+-----+-----+
| name | id | age | pmname | index | name |
+-----+-----+-----+-----+-----+-----+
| 张三 | 1 | 15 | 篮球 | 1 | 篮球 |
| 李四 | 2 | 15 | 排球 | 2 | 排球 |
+-----+-----+-----+-----+-----+-----+

```

体会inner join的操作

Inner join的细节

```

MariaDB [self_test]> select * from testtable;
+-----+-----+-----+-----+
| name | id | age | pmname |
+-----+-----+-----+-----+
| 张三 | 1 | 15 | 篮球 |
| 李四 | 2 | 15 | 排球 |
+-----+-----+-----+-----+

MariaDB [self_test]> select * from testtable inner
join sportstable on testtable.pmname
=sportstable.pmname;
+-----+-----+-----+-----+-----+-----+
| name | id | age | pmname | index | pmname |
+-----+-----+-----+-----+-----+-----+
| 张三 | 1 | 15 | 篮球 | 1 | 篮球 |
| 李四 | 2 | 15 | 排球 | 2 | 排球 |
+-----+-----+-----+-----+-----+-----+

```

```

MariaDB [self_test]> select name,id,age,pmname,index from testtable inner join sportstable on
testtable.pmname=sportstable.pmname;

```

ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near 'index from testtable inner join sportstable on testtable.pmname=sportstable.pmn' at line 1

```

MariaDB [self_test]> select name,id,age,testtable.pmname,index from testtable inner join
sportstable on testtable.pmname=sportstable.pmname;

```

ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MariaDB server version for the right syntax to use near 'index from testtable inner join sportstable on testtable.pmname=sportstable.pmn' at line 1

```

MariaDB [self_test]> select
testtable.name,testtable.id,testtable.age,testtable.pmname,sportstable.index from testtable
inner join sportstable on testtable.pmname=sportstable.pmname;

```

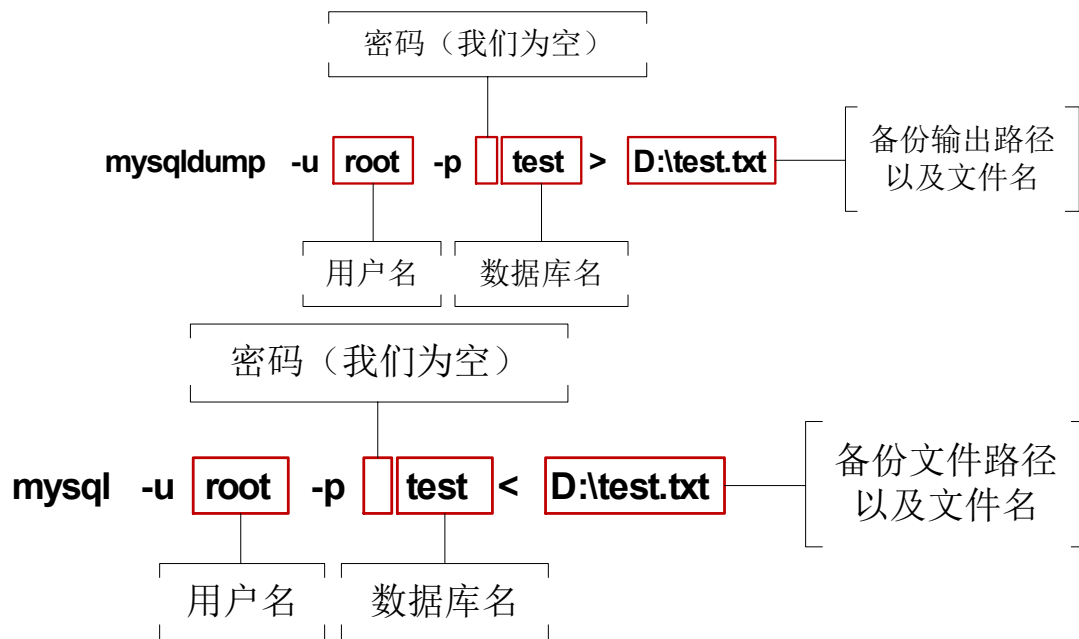
```

+-----+-----+-----+-----+-----+
| name | id | age | pmname | index |
+-----+-----+-----+-----+-----+
| 张三 | 1 | 15 | 篮球 | 1 |
| 李四 | 2 | 15 | 排球 | 2 |
+-----+-----+-----+-----+-----+

```

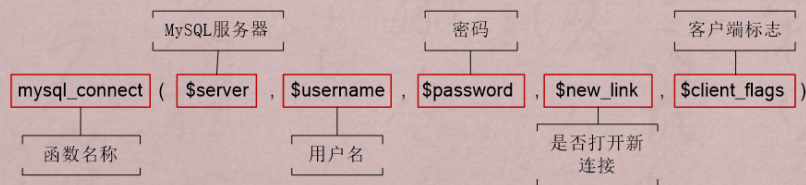
由查询结果可以看出，及时两个表有同名列且值相同，inner join也不会将它们合二为一；且经inner join连接后的表要查对应列的值必须要用原来的表.某列的方式，否则便会报错。

数据库备份（上）与恢复（下）



1. 连接数据库

- + 在PHP中使用`mysql_connect()`来与数据库服务器建立连接，它的语法如图所示。
- + 该函数的所有参数都是可选的。函数在成功连接数据库后返回一个MySQL连接标识符，连接失败则返回FALSE。`$client_flags`可以是以下参数的集合：
- + `MYSQL_CLIENT_COMPRESS`：使用压缩的通讯协议。
- + `MYSQL_CLIENT_IGNORE_SPACE`：允许在函数名后留空格位
- + `MYSQL_CLIENT_INTERACTIVE`：允许设置断开连接之前所空闲等候的`interactive_timeout`时间（代替`wait_timeout`）。
- + `MYSQL_CLIENT_SSL`：使用SSL加密。



`mysql1.php` 和 `clienthint.js` 以及 `gethint1.php` 配合使用：

mysql1.php:

```
<script src="clienthint.js?abcjd=aaaa"></script>
```

```
<script language="javascript">
```

```
function efg()
{
    var str=document.form1.usermobile.value;
    var str2=document.form1.useremail.value;
    var myreg=/^[1][3,4,5,7,8,9][0-9]{9}$/;
    var reg=new RegExp(/^( [a-zA-Z0-9._-])+@([a-zA-Z0-9_-])+(\.[a-zA-Z0-9_-])+/);
    if (!myreg.test(str)) {
        alert("请填写正确的手机号码");
        document.form1.usermobile.focus();
        return false;
    }
    if (!reg.test(str2)) {
        alert("请填写正确的邮箱地址");
        document.form1.useremail.focus();
        return false;
    }
    showHint(document.form1.userid.value);

    //document.form1.submit();
}
```

```
function abc(s)
{
    var checkboxes;
    var i;
    //var str = [];
    var str='';
    checkboxes = document.getElementsByName("checkbox0");
    for(i=0;i<checkboxes.length;i++){
        if(checkboxes[i].checked){
            //str.push(checkboxes[i].value);
            str = str + checkboxes[i].value + ',';
        }
    }
    document.form1.userinfo.value=str.substring(0,str.length-1);
    if (document.form1.userinfo.value=='')
    {
```

```

        document.form1.userinfo.value=s;
    }
    alert(document.form1.userinfo.value);
    //return false;
    document.form1.submit();
}
</script>
<?php
header("Cache-Control: no-cache, post-check=0, pre-check=0");
header("Content-type:text/html;charset=gb2312");
$userinfo=$_POST["userinfo"];

$con = mysql_connect("localhost","root","sql");
mysql_select_db("qing_zhou",$con); mysql_query("set names gb2312 ");
$userid=$_POST["userid"]; $upassword=$_POST["upassword"];
$username=$_POST["username"];
$userage=$_POST["userage"]; $usermobile=$_POST["usermobile"];
$useremail=$_POST["useremail"];

$sql = "select count(*) from php_admin where userid='$userid'";
$rs0 = mysql_query($sql);
$rs = mysql_fetch_array($rs0);
if ($rs[0]>=1)
{
    echo "用户 ID 重复, 请重新输入!";
    die();
}

$sql = "insert into
php_admin(userid,password,username,userage,usermobile,useremail)"
$sql = $sql . "
values('$userid','$upassword','$username',$userage,'$usermobile','$useremail')";
//echo $sql; //die();
$rs0 = mysql_query($sql);

$sql = "delete from php_admin where userid in (".userinfo.")";
//echo $sql; die();
$rs0 = mysql_query($sql);
$sql = "select * from php_admin where usertype=0";
$rs0 = mysql_query($sql); ?>
<form id=form1 name=form1 method=POST action="mysql1.php">
<input type=hidden id=userinfo name=userinfo> type="hidden" 创建了一个
隐藏的输入字段, 该字段不会在页面上显示给用户, 但它的值会随着表单一起提交到服务器。

```

```

<table>
<tr><td>用户 ID:</td><td><input type=text id=userid
name=userid></td></tr>
<tr><td>密码:</td><td><input type=password id=upassword
name=upassword></td></tr>
<tr><td>用户姓名:</td><td><input type=text id=username
name=username></td></tr>
<tr><td>用户年龄:</td><td><input type=text id=userage
name=userage></td></tr>
<tr><td>手机号:</td><td><input type=text id=usermobile
name=usermobile></td></tr>
<tr><td>EMAIL:</td><td><input type=text id=useremail
name=useremail></td></tr>
</table>

```

```

<table>
<CAPTION align= center>用户信息表</CAPTION>
  <TR><TH></TH><TH>用户 id</TH> <TH>用户类型</TH> <TH>密码</TH> <TH>姓名
</TH> <TH>年龄</TH> <TH>mobile</TH> <TH>email</TH></TR>
<?php
  $i=0;
  while($RS = mysql_fetch_array($RS0))//mysql_fetch_array() 每次调用只会
  从结果集中取出一行数据。
  // 通过 while 循环,可以逐行遍历整个结果集。
  // 每次循环,mysql_fetch_array() 会自动移动结果集的内部指针到下一行,直到所有
  行都被处理完毕。
  {  $i++; ?>
  <tr>
  <!--
  <td><input type="checkbox" id=checkbox<?php echo strval($i);?>
    name=checkbox<?php echo strval($i);?></td>
  -->
  <td><input type="checkbox" id=checkbox0 name=checkbox0
    value="'<?php echo $RS["userid"] ?>'"></td>
  <td align=left>&nbsp;<?php echo $RS["userid"]?></td>
  <td align=left>&nbsp;<?php echo $RS["usertype"]?></td>
  <td align=left>&nbsp;<?php echo $RS["password"]?></td>
  <td align=left>&nbsp;<?php echo $RS["username"]?></td>
  <td align=left>&nbsp;<?php echo $RS["userage"]?></td>
  <td align=left>&nbsp;<?php echo $RS["usermobile"]?></td>
  <td align=left>&nbsp;<?php echo $RS["useremail"]?></td>

```

```

<td align=left><input type=button value="删除" onclick="return
abc('<?php echo $RS["userid"]?>');"> </td>

</tr>
<?php }
mysql_close($con); $RS=NULL; $Con =NULL;
?>
<tr><td colspan=7 align=left><input type=button value="删除"
onclick="return abc();"
<input type=button value="保存" onclick="return efg();"> </td></tr>
<tr><td colspan=7 align=left><span id="txtHint"
name="txtHint"></span></td></tr>
</table>

</form>

```

clienthint.js:

```

/*
showHint() 函数
每当在输入域中输入一个字符，该函数就会被执行一次。

如果文本框中有内容 (str.length > 0)，该函数这样执行：

定义要发送到服务器的 URL（文件名）
把带有输入域内容的参数 (q) 添加到这个 URL
添加一个随机数，以防服务器使用缓存文件
调用 GetXmlHttpRequest 函数来创建 XMLHttpRequest 对象，
并在事件被触发时告知该对象执行名为 stateChanged 的函数
用给定的 URL 来打开打开这个 XMLHttpRequest 对象
向服务器发送 HTTP 请求
如果输入域为空，则函数简单地清空 txtHint 占位符的内容。

```

```

*/

var xmlhttp;

function showHint(str)
{
    if (str.length==0)
    {
        document.getElementById("txtHint").innerHTML=""
        return
    }
}

```

```

xmlHttp=GetXmlHttpRequest()
if (xmlHttp==null)
{
    alert ("Browser does not support HTTP Request")
    return
}

var url="gethint1.php";
url=url+"?q="+str;
url=url+"&sid="+Math.random();
xmlHttp.onreadystatechange=stateChanged ;
xmlHttp.open("GET",url,true);
xmlHttp.send(null);

//document.getElementById("txtHint").innerHTML=xmlHttp.responseText;

}

```

/*

stateChanged() 函数

每当 XMLHttpRequest 对象的状态发生改变，
则执行该函数。

在状态变成 4（或 "complete"）时，
用响应文本填充 txtHint 占位符 txtHint 的内容。

*/

/*

发送一个请求后，
客户端无法确定什么时候
会完成这个请求，
所以需要用事件机制来捕获请求的状态，
XMLHttpRequest 对象提供了
onreadystatechange 事件实现这一功能。
这类似于回调函数的做法。
onreadystatechange
事件可指定一个事件处理函数来处理
XMLHttpRequest 对象的执行结果
onreadystatechange 事件是在
readyState 属性发生改变时触发的，

readyState 的值表示了当前请求的状态，
在事件处理程序中可以根据这个值
来进行不同的处理。

readyState 有五种可取值

- 0: 尚未初始化,
- 1: 正在加载,
- 2: 加载完毕,
- 3: 正在处理;
- 4: 处理完毕。

一旦 readyState 属性的值变成了 4,
就可以从服务器返回的响应数据
进行访问了。

*/

```
function stateChanged()  
{  
    var str;  
    if (xmlHttp.readyState==4 || xmlHttp.readyState=="complete")  
    {  
        str= xmlHttp.responseText;  
        if (str=="1")  
        {  
            alert('userid 重复! ');  
            document.form1.userid.focus();  
            return false;  
        }  
        else if (str=="0")  
        {  
            document.form1.submit();  
        }  
    }  
}
```

/*

GetXmlHttpRequest() 函数

AJAX 应用程序只能运行在完整支持 XML 的 web 浏览器中。

上面的代码调用了名为 GetXmlHttpRequest() 的函数。

该函数的作用是解决为不同浏览器创建不同 XMLHttpRequest 对象的问题。

*/

```
function GetXmlHttpRequest()  
{  
    var xmlhttp=null;  
    try  
    {
```

```

// Firefox, Opera 8.0+, Safari
xmlHttp=new XMLHttpRequest();
}
catch (e)
{
// Internet Explorer
try
{
xmlHttp=new ActiveXObject("Msxml2.XMLHTTP");
}
catch (e)
{
xmlHttp=new ActiveXObject("Microsoft.XMLHTTP");
}
}
return xmlHttp;
}

```

gethint1.php:

```

<?php
$con = mysql_connect("localhost","root","sql");
mysql_select_db("qing_zhou",$con); mysql_query("set names gb2312 ");
$q=$_GET["q"];

$sql="select count(*) from php_admin where userid='$q'";
$rs0 = mysql_query($sql);
$rs = mysql_fetch_array($rs0);
if ($rs[0]>=1)
{
    echo "1";
}
else
{
    echo "0";
}
mysql_close($con); $rs=NULL; $con=NULL;
?>

```

用户ID:

密码:

用户姓名:

用户年龄:

手机号:

EMAIL:

用户信息表

用户id	用户类型	密码	姓名	年龄	mobile	email	
☐ 1	0	1111111	张飞	11	15458956230	154623075@cc.cc	删除
☐ 2	0	123456789	刘备	49	15462307531	15623075@cc.com	删除

删除 保存

★ 书签

📖 手机书签

📍 网址导航

🔍 DeepSeek - 探索...

📄 PPT精华教程合集...

🌐 天

localhost 显示

哔哩哔哩 (' - ') ...

用户ID:

密码:

用户姓名:

用户年龄:

手机号:

EMAIL:

用户信息表

用户id	用户类型	密码	姓名	年龄	mobile	email	
☐ 1	0	1111111	张飞	11	15458956230	154623075@cc.cc	删除
☐ 2	0	123456789	刘备	49	15462307531	15623075@cc.com	删除

删除 保存

点击删除的效果

```
<TABLE border=1>
<CAPTION align=center>用户信息表</CAPTION>
<TR><TH></TH><TH>用户 id</TH> <TH>姓名</TH> <TH>密码</TH> <TH>年龄
</TH><TH>电话</TH><TH>EMAIL</TH><TH></TH></TR>

<?php
while($RS = mysql_fetch_array($RS0))
{
?>
<TR>
<td><input type="checkbox" id=checkbox0 name=checkbox0 value="1"><?php
echo $RS["userid"] ?>"></td>
<TD><?php echo $RS["userid"] ?> </TD>
<TD><?php echo $RS["username"] ?> </TD>
<TD><?php echo $RS["password"] ?> </TD>
<TD><?php echo $RS["userage"] ?> </TD>
<TD><?php echo $RS["usermobile"] ?> </TD>
<TD><?php echo $RS["useremail"] ?> </TD>
<TD><input type=button value="删除" onclick="return abc('<?php echo
$RS["userid"]?>');"> </TD>
</TR>
<?php
}
mysql_close($con);
```

```
?>
```

```
</TABLE>
```

```
<input type=button value="保存" onclick="return efg();">
```

```
<input type=button value="删除" onclick="return abc();">
```

将对应处的代码替换成上面的样式，可以得到下面示例的网页输出效果：

用户ID:

密码:

用户姓名:

用户年龄:

手机号:

EMAIL:

用户信息表

	用户id	姓名	密码	年龄	电话	EMAIL	
☐	11	刘备	1111	11	13546892155	154623@cc.com	删除

保存 删除

loginform.php 和 logcheck.php 配合使用：主要看他是怎么用 session 变量和怎么用定界符来表示字符串的。

loginform.php:

```
<?
```

```
header("Content-type: text/html; charset=GB2312");
```

```
?>
```

```
<form name="fmlog" method="post" action="logcheck.php?para=para1">
```

用户名:<input type="text" name="username">

密码:<input type="password" name="password">


```
<input type="hidden" name="action" value="login">
```

```
<input type="submit" value="登录">
```

```
</form>
```

logcheck.php:

```
<?php
```

```
global $c;
```

```
$c = 0;
```

```
//isset() 函数用于检测变量是否已设置并且非 NULL。
```

```
//区别不同提交方式得到的变量
```

```
if ( isset($_POST["action"]) && $_POST["action"] == "login"){
```

```
    // 处理登录的代码
```

```
    $user=$_POST["username"];
```

```

    $pwd=$_POST["password"];
    $para=$_GET["para"];

    // 记录用户 id 和 password
    session_start();
    $_SESSION["USERID"] = strtoupper($user);
    $_SESSION["USERPSW"] = $pwd;

    echo $_SESSION["USERID"] . $para . " logged in. Your are welcome.
";
    echo '<form name="fmlog" method="post">';
    echo '<input type="hidden" name="action" value="logout" >';
    echo '<input type="submit" value="注销">';
    echo '</form>';

print <<<EOT
{$_SESSION["USERID"]} logged in. Your are welcome.
<form name="fmlog" method="post">
<input type="hidden" name="action" value="logout" >
<input type="submit" value="注销">
</form>
EOT;
}
?>

```

Demo1.html、clienthint.js、gethint.php:

Demo1.html

```
<!--
```

该表单是这样工作的:

当用户在输入域中按下并松开按键时，
 会触发一个事件
 当该事件被触发时，执行名为 showHint() 的函数
 表单的下面是一个名为 "txtHint" 的 。
 它用作 showHint() 函数所返回数据的占位符。
 -->

```

<html>
<head>
<script src="clienthint.js?abmcjd=gf"></script>

</head>

<body>

```

```

<form id="form1">
First Name:
<input type="text" id="txt1" onkeyup="showHint(this.value)">

</form>

<p>Suggestions: <span id="txtHint" name="txtHint">abc</span></p>
</body>
</html>

```

clienthint.js:

```

/*
showHint() 函数
每当在输入域中输入一个字符，该函数就会被执行一次。

如果文本框中有内容 (str.length > 0)，该函数这样执行：

定义要发送到服务器的 URL（文件名）
把带有输入域内容的参数 (q) 添加到这个 URL
添加一个随机数，以防服务器使用缓存文件
调用 GetXmlHttpRequest 函数来创建 XMLHttpRequest 对象，
并在事件被触发时告知该对象执行名为 stateChanged 的函数
用给定的 URL 来打开打开这个 XMLHttpRequest 对象
向服务器发送 HTTP 请求
如果输入域为空，则函数简单地清空 txtHint 占位符的内容。

*/

var xmlhttp;

function showHint(str)
{
    if (str.length==0)
    {
        document.getElementById("txtHint").innerHTML=""
        //document.txtHint.innerHTML=""
        return
    }
    xmlhttp=GetXmlHttpRequest()
    if (xmlhttp==null)
    {
        alert ("Browser does not support HTTP Request")
        return
    }

```



```

    }

    var url="gethint.php";
    url=url+"?q="+str;
    url=url+"&sid="+Math.random();
    xmlhttp.onreadystatechange=stateChanged;
    xmlhttp.open("GET",url,true);
    xmlhttp.send(null);
    //document.getElementById("txtHint").innerHTML=xmlhttp.responseText;
    如果把 xmlhttp.onreadystatechange=stateChanged; 注释了，然后把该函数注释取消，那么就有可能跑不出推荐结果
    //跑的出来的情况：当前的网址不变，浏览器中保存了原 js 文件，相当于没有改动代码，浏览器执行的还是之前的代码，这样就能跑出来
    //跑不出来的情况，网址改变，前端会重新加载 js 文件，而更改后的代码，不会有回调函数，js 代码执行该代码
    document.getElementById("txtHint").innerHTML=xmlhttp.responseText 的时间比浏览器从服务器拿到 resonsetext 的时间短，故等代码执行完后浏览器拿不到数据就不会生成推荐
}

/*
stateChanged() 函数
每当 XMLHttpRequest 对象的状态发生改变，
则执行该函数。

在状态变成 4（或 "complete"）时，
用响应文本填充 txtHint 占位符 txtHint 的内容。
*/

/*
发送一个请求后，
客户端无法确定什么时候
会完成这个请求，
所以需要用事件机制来捕获请求的状态，
XMLHttpRequest 对象提供了
onreadystatechange 事件实现这一功能。
这类似于回调函数的做法。
onreadystatechange
事件可指定一个事件处理函数来处理
XMLHttpRequest 对象的执行结果
onreadystatechange 事件是在
readyState 属性发生改变时触发的，

readyState 的值表示了当前请求的状态，

```

在事件处理程序中可以根据这个值来进行不同的处理。

readyState 有五种可取值

- 0: 尚未初始化,
- 1: 正在加载,
- 2: 加载完毕,
- 3: 正在处理;
- 4: 处理完毕。

一旦 readyState 属性的值变成了 4, 就可以从服务器返回的响应数据进行访问了。

*/

```
function stateChanged()  
{  
if (xmlHttp.readyState==4 || xmlHttp.readyState=="complete")  
{  
    document.getElementById("txtHint").innerHTML= xmlHttp.responseText;  
}  
}
```

/*

GetXmlHttpRequest() 函数

AJAX 应用程序只能运行在完整支持 XML 的 web 浏览器中。

上面的代码调用了名为 GetXmlHttpRequest() 的函数。

该函数的作用是解决为不同浏览器创建不同 XMLHttpRequest 对象的问题。

*/

```
function GetXmlHttpRequest()  
{  
var xmlHttp=null;  
try  
{  
    // Firefox, Opera 8.0+, Safari  
    xmlHttp=new XMLHttpRequest();  
}  
catch (e)  
{  
    // Internet Explorer  
    try  
    {  
        xmlHttp=new ActiveXObject("Msxml2.XMLHTTP");  
    }  
    }  
}
```

```

    }
    catch (e)
    {
        xmlHttp=new ActiveXObject("Microsoft.XMLHTTP");
    }
}
return xmlHttp;
}

```

//考试时创建 AJAX 对象时直接抄这个代码

gethint.php

```

<?php
// Fill up array with names
$a[]="Anna";
$a[]="Brittany";
$a[]="Cinderella";
$a[]="Diana";
$a[]="Eva";
$a[]="Fiona";
$a[]="Gunda";
$a[]="Hege";
$a[]="Inga";
$a[]="Johanna";
$a[]="Kitty";
$a[]="Linda";
$a[]="Nina";
$a[]="Ophelia";
$a[]="Petunia";
$a[]="Amanda";
$a[]="Raquel";
$a[]="Cindy";
$a[]="Doris";
$a[]="Eve";
$a[]="Evita";
$a[]="Sunniva";
$a[]="Tove";
$a[]="Unni";
$a[]="Violet";
$a[]="Liza";
$a[]="Elizabeth";
$a[]="Ellen";
$a[]="Wenche";
$a[]="Vicky";

//get the q parameter from URL

```

```

$q=$_GET["q"];

//lookup all hints from array if length of q>0
if (strlen($q) > 0)
{
$hint="";
for($i=0; $i<count($a); $i++)
{
if (strtolower($q)==strtolower(substr($a[$i],0,strlen($q))))
{
if ($hint=="")
{
$hint=$a[$i];
}
else
{
$hint=$hint." , ".$a[$i];
}
}
}
}

//Set output to "no suggestion" if no hint were found
//or to the correct values
if ($hint == "")
{
$response="no suggestion";
}
else
{
$response=$hint;
}

//output the response
echo $response;
?>

```

demo2.html、selectcd.js、getcd.php:

demo2.html

```

<html>
<head>
<script src="selectcd.js?a=2"></script>
</head>

```

```

<body>

<form>
Select a CD:
<select name="cds" onchange="showCD(this.value)">
<option value="Bob Dylan">Bob Dylan</option>
<option value="Bee Gees">Bee Gees</option>
<option value="Cat Stevens">Cat Stevens</option>
</select>
</form>

<p>
<div id="txtHint"><b>CD info will be listed here.</b></div>
</p>

</body>
</html>

```

selectcd.js

```

var xmlhttp

function showCD(str)
{
xmlhttp=GetXmlHttpRequest()
if (xmlhttp==null)
{
    alert ("Browser does not support HTTP Request")
    return
}
var url="getcd.php"
url=url+"?q="+str
url=url+"&sid="+Math.random()
xmlhttp.onreadystatechange=stateChanged
xmlhttp.open("GET",url,true)
xmlhttp.send(null)
}

function stateChanged()
{
    if (xmlhttp.readyState==4 || xmlhttp.readyState=="complete")
    {
        document.getElementById("txtHint").innerHTML=xmlhttp.responseText
    }
}

```

```

function GetXmlHttpRequest()
{
var xmlhttp=null;

try
{
// Firefox, Opera 8.0+, Safari
xmlhttp=new XMLHttpRequest();
}
catch (e)
{
// Internet Explorer
try
{
xmlhttp=new ActiveXObject("Msxml2.XMLHTTP");
}
catch (e)
{
xmlhttp=new ActiveXObject("Microsoft.XMLHTTP");
}
}
return xmlhttp;
}
//创建 ajax 对象

```

Getcd.php

```
<?php
```

```
/*
```

当请求从 JavaScript 发送到 PHP 页面时，发生：

PHP 创建 "cd_catalog.xml" 文件的 XML DOM 对象

循环所有 "artist" 元素 (nodetypes = 1)，

查找与 JavaScript 所传数据相匹配的名字

找到 CD 包含的正确 artist

输出 album 的信息，并发送到 "txtHint" 占位符

```
*/
```

```
/*
```

节点

根据 DOM，XML 文档中的每个成分都是一个节点。

DOM(document) 是这样规定的：

整个文档是一个文档节点

每个 XML 标签是一个元素节点

包含在 XML 元素中的文本是文本节点

每一个 XML 属性是一个属性节点

注释属于注释节点

nodeType 属性可返回节点的类型。

最重要的节点类型是：

元素类型 节点类型

元素 element 1

属性 attr 2

文本 text 3

注释 comments 8

文档 document 9

*/

```
$q=$_GET["q"];
```

```
$xmlDoc = new DOMDocument();
```

```
$xmlDoc->load("cd_catalog.xml");
```

```
//返回 xmlDoc 文档节点下的所有 <ARTIST> 元素节点
```

```
$x=$xmlDoc->getElementsByTagName('ARTIST');
```

```
for ($i=0; $i<=$x->length-1; $i++)
```

```
{
```

```
//Process only element nodes
```

```
if ($x->item($i)->nodeType==1)
```

```
{
```

```
// 或者 if ($x->item($i)->childNodes->item(0)->nodeValue == $q)
```

```
//echo $x->item($i)->nodeValue." : ".$x->item($i)->nodeName."<br>";
```

```
if ($x->item($i)->nodeValue == $q)
```

```
{
```

```
$y=($x->item($i)->parentNode);
```

```
}
```

```
}
```

```
}
```

```
$cd=($y->childNodes);
```

```
//echo $cd->length;
```

```
for ($i=0;$i<$cd->length;$i++)
```

```
{
```

```
//Process only element nodes
```

```
if ($cd->item($i)->nodeType==1)
```

```
{
```

```
echo($cd->item($i)->nodeName);
```

```
echo(" : ");
```

```
echo($cd->item($i)->nodeValue);
```

```
    echo("<br>");  
}  
?  
?>
```

xml 文件记录的例子:

```
<CATALOG>  
  
    <CD>  
  
        <TITLE>Empire Burlesque</TITLE>  
  
        <ARTIST>Bob Dylan</ARTIST>  
  
        <COUNTRY>USA</COUNTRY>  
  
        <COMPANY>Columbia</COMPANY>  
  
        <PRICE>10.90</PRICE>  
  
        <YEAR>1985</YEAR>  
  
    </CD>  
  
</CATALOG>
```

平时作业

九九乘法表:

```
<?php  
  
for ($i=1;$i<=9;$i++){  
    for ($j=1;$j<=9;$j++){  
        if ($j>$i) {break;}  
        $k=$i*$j;  
        echo $i."*".$j."=".$k." ";  
        if ($j==$i) {echo "<br>";}  
    }  
}  
?  
?>
```

SQL 有 qing zhou 数据库，有学生表 t_student，课程表 t_course，选课表 t_score0，各表的字段含义如下所示，采用 SQL 语句作答（共 20 分，每个小题各 5 分）。

表名	字段名	含义	类型	备注
t_student	stucode	学号	char(10)	主键
	stuname	姓名	varchar(255)	
	stuage	年龄	int	
	stuclass	班级	varchar(255)	
t_course	courseid	课程 ID	int	主键，自动增长
	coursename	课程名称	varchar(255)	
	coursehour	学时	tinyint	
	t_score0	分数	int	
t_score0	stucode	学号	char(10)	主键
	courseid	课程 ID	int	主键
	score	分数	int	

(一) 列出姓“王”且年龄不小于 21 岁的学生，输出学生表四个字段。

(二) 新增一名学生，信息是“m202301001”，“王二”，20，“AC202301”，写出新增语句。

(三) 教师想给上了“电子商务”，名字叫“王二”的学生修改成绩，写出修改分数为 90 分的一条语句。

答案：

(一) 列出姓“王”且年龄不小于 21 岁的学生，输出学生表四个字段。

```

Select * from t_student where stuname like '王%' and stuage >= 21

```

(二) 新增一名学生，信息是“m202301001”，“王二”，20，“AC202301”，写出新增语句。

```

Insert into t_student values('m202301001', '王二', 20, 'AC202301')

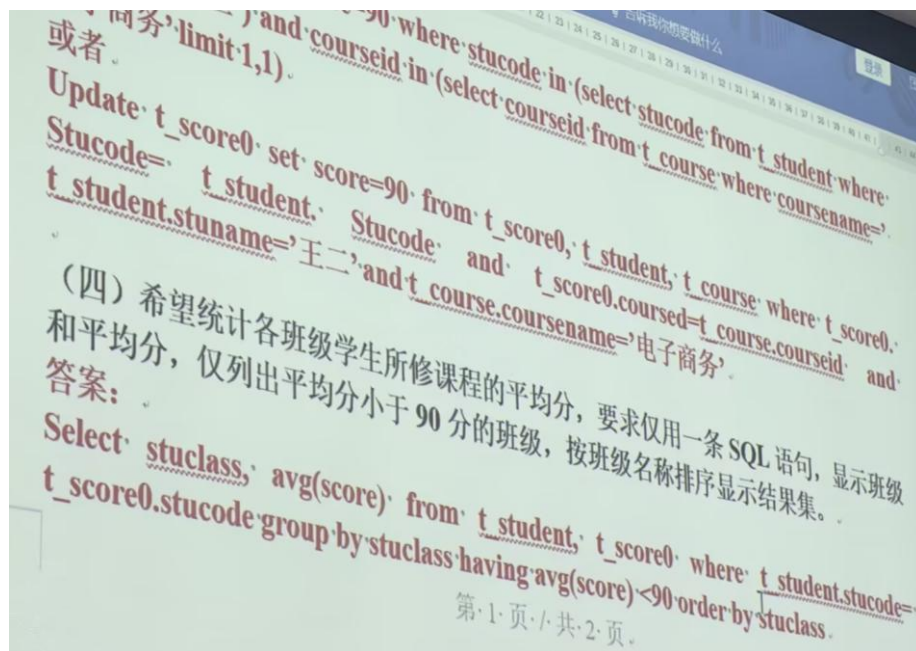
```

(三) 教师想给上了“电子商务”，名字叫“王二”的学生修改成绩，写出修改分数为 90 分的一条语句。

```

Update t_score0 set score=90 where stucode in (select stucode from t_student where stuname='王二') and courseid in (select courseid from t_course where coursename='电子商务' limit 1,1)
或者
Update t_score0 set score=90 where stucode in (select stucode from t_student where stuname='王二') and courseid in (select courseid from t_course where coursename='电子商务' limit 1,1)

```



Userreg.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>用户界面</title>
  <style>
  </style>
</head>
<body>
<form id=form1 name=form1 method=POST action="userinsert.php"
enctype="multipart/form-data">
<input type=hidden id=userinfo name=userinfo>
<table>
<tr><td>用户名:</td><td><input type=text id=username
name=username></td></tr>
<tr><td>密码:</td><td><input type=password id=upassword
name=upassword></td></tr>
<tr><td>年龄:</td><td><input type="text" id=userage
name=userage></td></tr>
<tr><td>手机号:</td><td><input type=text id=usermobile
name=usermobile></td></tr>
<tr><td>头像:</td><td><input type="file" value="选择文件" id=userphoto
name=userphoto></td></tr>
```

```

<tr><td align=left><input type=button value="保存" onclick="return
efg() &&
showHint(document.form1.username.value,document.form1.usermobile.valu
e);" > </td></tr>
<tr><td colspan="2" class="hint" id="txtHint"></td></tr>
</table>
<script src="Clienthint.js?abcjd=aa"></script>
<script language="javascript">

```

```

    function efg()
    {
        var str=document.form1.usermobile.value;
        var myreg=/^[1][3,4,5,7,8,9][0-9]{9}$/;
        if (!myreg.test(str)) {
            alert("请填写正确的手机号码");
            document.form1.usermobile.focus();
            return false;
        }
        return true;
    }
</script>
</body>
</html>

```

Clienthint.js

```

/*
showHint() 函数
每当在输入域中输入一个字符，该函数就会被执行一次。

```

如果文本框中有内容 (str.length > 0)，该函数这样执行：

定义要发送到服务器的 URL（文件名）
 把带有输入域内容的参数（q）添加到这个 URL
 添加一个随机数，以防服务器使用缓存文件
 调用 GetXmlHttpRequest 函数来创建 XMLHttpRequest 对象，
 并在事件被触发时告知该对象执行名为 stateChanged 的函数
 用给定的 URL 来打开打开这个 XMLHttpRequest 对象
 向服务器发送 HTTP 请求
 如果输入域为空，则函数简单地清空 txtHint 占位符的内容。

```

*/

```

```

var xmlHttp;

```

```

function showHint(str0,str1)
{
    var hintElem = document.getElementById("txtHint");
    hintElem.innerHTML = "";
    if (str0.length === 0 && str1.length === 0 ) {
        return;
    }
    xmlhttp=GetXmlHttpRequest()
    if (xmlhttp==null)
    {
        alert ("Browser does not support HTTP Request")
        return
    }

    var url="usercheck.php";
    url=url+"?name="+str0;
    url=url+"&mobile="+str1;
    url=url+"&sid="+Math.random();
    xmlhttp.onreadystatechange=stateChanged ;
    xmlhttp.open("GET",url,true);
    xmlhttp.send(null);

    //document.getElementById("txtHint").innerHTML=xmlhttp.responseText;

}

```

/*

stateChanged() 函数

每当 XMLHttpRequest 对象的状态发生改变，
则执行该函数。

在状态变成 4 （或 "complete"）时，
用响应文本填充 txtHint 占位符 txtHint 的内容。

*/

/*

发送一个请求后，
客户端无法确定什么时候
会完成这个请求，
所以需要用事件机制来捕获请求的状态，
XMLHttpRequest 对象提供了
onreadystatechange 事件实现这一功能。
这类似于回调函数的做法。
onreadystatechange

事件可指定一个事件处理函数来处理
XMLHttpRequest 对象的执行结果
onreadystatechange 事件是在
readyState 属性发生改变时触发的，

readyState 的值表示了当前请求的状态，
在事件处理程序中可以根据这个值
来进行不同的处理。

readyState 有五种可取值

- 0：尚未初始化，
- 1：正在加载，
- 2：加载完毕，
- 3：正在处理；
- 4：处理完毕。

一旦 readyState 属性的值变成了 4，
就可以从服务器返回的响应数据
进行访问了。

*/

```
function stateChanged()
{
    var hintElem = document.getElementById("txtHint");
    var hint = "";
    if (xmlHttp.readyState==4 || xmlHttp.readyState=="complete")
    {
        const response = xmlHttp.responseText.trim();
        const [varify0, varify1] = response.split(",");
        if (varify0=="1")
        {
            hint += "用户名重复，请重新输入! <br>";
        }
        if (varify1=="1")
        {
            hint += "用户号码重复，请重新输入! <br>";
        }
        if (hint !== "")
        {
            hintElem.innerHTML = hint;
            return false;
        } else {
            // 验证通过，提交表单
            hintElem.innerHTML = "";
            document.getElementById("form1").submit();
            return true;
        }
    }
}
```



```

    }
}
}

```

```

/*

```

GetXmlHttpRequest() 函数

AJAX 应用程序只能运行在完整支持 XML 的 web 浏览器中。

上面的代码调用了名为 GetXmlHttpRequest() 的函数。

该函数的作用是解决为不同浏览器创建不同 XMLHttpRequest 对象的问题。

```

*/

```

```

function GetXmlHttpRequest()
{
var xmlhttp=null;
try
{
// Firefox, Opera 8.0+, Safari
xmlhttp=new XMLHttpRequest();
}
catch (e)
{
// Internet Explorer
try
{
xmlhttp=new ActiveXObject("Msxml2.XMLHTTP");
}
catch (e)
{
xmlhttp=new ActiveXObject("Microsoft.XMLHTTP");
}
}
return xmlhttp;
}

```

usercheck.php

```

/*

```

showHint() 函数

每当在输入域中输入一个字符，该函数就会被执行一次。

如果文本框中有内容 (str.length > 0)，该函数这样执行：

定义要发送到服务器的 URL (文件名)

把带有输入域内容的参数 (q) 添加到这个 URL

添加一个随机数，以防服务器使用缓存文件
调用 GetXmlHttpRequest 函数来创建 XMLHttpRequest 对象，
并在事件被触发时告知该对象执行名为 stateChanged 的函数
用给定的 URL 来打开打开这个 XMLHttpRequest 对象
向服务器发送 HTTP 请求
如果输入域为空，则函数简单地清空 txtHint 占位符的内容。

```
*/  
  
var xmlHttp;  
  
function showHint(str0,str1)  
{  
    var hintElem = document.getElementById("txtHint");  
    hintElem.innerHTML = "";  
    if (str0.length === 0 && str1.length === 0 ) {  
        return;  
    }  
    xmlHttp=GetXmlHttpRequest()  
    if (xmlHttp==null)  
    {  
        alert ("Browser does not support HTTP Request")  
        return  
    }  
  
    var url="usercheck.php";  
    url=url+"?name="+str0;  
    url=url+"&mobile="+str1;  
    url=url+"&sid="+Math.random();  
    xmlHttp.onreadystatechange=stateChanged ;  
    xmlHttp.open("GET",url,true);  
    xmlHttp.send(null);  
  
    //document.getElementById("txtHint").innerHTML=xmlHttp.responseText;  
  
}  
  
/*  
stateChanged() 函数  
每当 XMLHttpRequest 对象的状态发生改变，  
则执行该函数。
```

在状态变成 4 （或 "complete"）时，

用响应文本填充 txtHint 占位符 txtHint 的内容。

```
*/
```

```
/*
```

发送一个请求后，
客户端无法确定什么时候
会完成这个请求，
所以需要用事件机制来捕获请求的状态，
XMLHttpRequest 对象提供了
onreadystatechange 事件实现这一功能。
这类似于回调函数的做法。
onreadystatechange
事件可指定一个事件处理函数来处理
XMLHttpRequest 对象的执行结果
onreadystatechange 事件是在
readyState 属性发生改变时触发的，

readyState 的值表示了当前请求的状态，
在事件处理程序中可以根据这个值
来进行不同的处理。

readyState 有五种可取值

- 0：尚未初始化，
- 1：正在加载，
- 2：加载完毕，
- 3：正在处理；
- 4：处理完毕。

一旦 readyState 属性的值变成了 4，
就可以从服务器返回的响应数据
进行访问了。

```
*/
```

```
function stateChanged()  
{  
    var hintElem = document.getElementById("txtHint");  
    var hint = "";  
    if (xmlHttp.readyState==4 || xmlHttp.readyState=="complete")  
    {  
        const response = xmlHttp.responseText.trim();  
        const [varify0, varify1] = response.split(",");  
        if (varify0=="1")  
        {  
            hint += "用户名重复，请重新输入！<br>";  
        }  
        if (varify1=="1")
```

```

    {
        hint += "用户号码重复，请重新输入！<br>";
    }
    if (hint !== "")
    {
        hintElem.innerHTML = hint;
        return false;
    } else {
        // 验证通过，提交表单
        hintElem.innerHTML = "";
        document.getElementById("form1").submit();
        return true;
    }
}
}
/*
GetXmlHttpRequest() 函数
AJAX 应用程序只能运行在完整支持 XML 的 web 浏览器中。

```

上面的代码调用了名为 `GetXmlHttpRequest()` 的函数。

该函数的作用是解决为不同浏览器创建不同 XMLHTTP 对象的问题。

```

*/
function GetXmlHttpRequest()
{
    var xmlHttp=null;
    try
    {
        // Firefox, Opera 8.0+, Safari
        xmlHttp=new XMLHttpRequest();
    }
    catch (e)
    {
        // Internet Explorer
        try
        {
            xmlHttp=new ActiveXObject("Msxml2.XMLHTTP");
        }
        catch (e)
        {
            xmlHttp=new ActiveXObject("Microsoft.XMLHTTP");
        }
    }
    return xmlHttp;
}

```

```
}
```

userinsert.php

```
<?php
include "save_img.php";
$con = mysql_connect("localhost","root","sql");
if (!$con) {
    die("数据库连接失败: " . mysql_error());
}
mysql_select_db("qing_zhou", $con); mysql_query("set names gb2312 ");
$password=$_POST["upassword"]; $username=$_POST["username"];
$age=$_POST["userage"]; $mobile=$_POST["usermobile"];
$sql = "insert into t_user(password,username,age,mobile,photo)";
$sql = $sql . "
values('$password','$username','$age','$mobile','$photo'
)";
//echo $sql; //die();
$rs = mysql_query($sql);
if ($rs)
{
    $userid = mysql_insert_id(); // 获取自增 ID
    echo "<table>";
    echo "<tr><td>用户 ID:</td><td>$userid</td></tr>";
    echo "<tr><td>用户名:</td><td>$username</td></tr>";
    echo "<tr><td>密码:</td><td>$password</td></tr>";
    echo "<tr><td>年龄:</td><td>$age</td></tr>";
    echo "<tr><td>手机号:</td><td>$mobile</td></tr>";
    echo "<tr><td>头像:</td><td><img src='$photo' width='100px'
height='100px'></td></tr>"; // 固定 100x100
    echo "</table>";
}
mysql_close($con);
$rs = NULL;
$con = NULL;
?>
```

save_img.php

```
<?php
$username = $_POST['username'];
$uploadDir = 'E:/xampp/www/upload/';
$uploadFile="";
$fileType="";
$userphoto="";
// 检查是否有文件上传
```

```
    if (isset($_FILES['userphoto']) && $_FILES['userphoto']['error']  
== UPLOAD_ERR_OK)  
    {  
        $fileType = strtolower(pathinfo($_FILES['userphoto']['name'],  
PATHINFO_EXTENSION));  
        $uploadFile =  
$uploadDir . $username . '.' . strtolower(pathinfo($_FILES['userphoto']['n  
ame'], PATHINFO_EXTENSION));  
        // 移动文件到 upload 目录  
        move_uploaded_file($_FILES['userphoto']['tmp_name'],  
$uploadFile);  
    }  
    $userphoto="$username"."."."$fileType";  
?>
```

B2C 案例

登录，验证，根据 usertype 给不同的页面



userlogin.htm

```
1  <html>
2
3  <head>
4  <meta http-equiv="Content-Type" content="text/html; charset=gb2312">
5  <title>用户登录</title>
6  </head>
7
8  <body>
9  <form action="userlogin.php" name="form1" method="post">
10  用户名: <input type="text" id="userid" name="userid">
11  密码: <input type="password" id="password" name="password">
12  <input type="submit" id="submit1" value="登录">
13  </form>
14
15 </body>
16
17 </html>
18
```

userlogin.php


```

1  <?php
2
3  header("Cache-Control: no-cache, post-check=0, pre-check=0");
4  header("Content-type:text/html;charset=gb2312");
5
6  session_start();
7  $_SESSION['userid']='';
8  $_SESSION['usertype']='';
9
10 $userid = $_POST['userid'];
11 $password = $_POST['password'];
12
13 $con = mysql_connect("localhost","root","sql");
14 mysql_select_db("qing_zhou", $con);
15 mysql_query("set names gb2312 ");
16
17 $sql1 = "select userid,usertype from php_admin where userid='".$userid.'" and password='".$password.'"";
18 //echo $sql1; die();
19 $RS0 = mysql_query($sql1, $con);
20
21 if (mysql_num_rows($RS0)>0)
22 {
23     $RS = mysql_fetch_array($RS0);
24     $_SESSION['userid'] = $RS['userid'];
25     $_SESSION['usertype'] = $RS['usertype'];
26     mysql_close($con);
27     $RS=NULL;
28     $con =NULL;
29     header("location: main.php");//跳转main.php
30 }
31 else
32 {
33     mysql_close($con);
34     $RS=NULL;
35     $con =NULL;
36     header("location: userlogin.htm");
37 }
38
39 ?>

```

main.php

```

1  <?php
2  session_start();
3  header("Cache-Control: no-cache, post-check=0, pre-check=0");
4  header("Content-type:text/html;charset=gb2312");
5  ?>
6
7  <html xmlns="http://www.w3.org/1999/xhtml" >
8
9
10 <head>
11 <meta HTTP-EQUIV="Content-Type" CONTENT="text/html; charset=gb2312">
12 <meta name="GENERATOR" content="Microsoft FrontPage 4.0">
13 <meta name="ProgId" content="FrontPage.Editor.Document">
14 <title>信息系统</title>
15 </head>
16
17 <frameset rows="10%,*">
18     <frame name="banner" scrolling="no" noresize target="contents" src="top.php">
19
20     <frameset cols="22%,*" frameborder="NO" border="1" framespacing="1" name="main" scrolling="yes">
21         <frame name="left" frameborder="yes" border="1" scrolling="auto" src="lefttree.php">
22         <frame name="right" noresize
23             <?php if ($_SESSION['usertype']=='0') { ?> src="selectproduct0.php" <?php }
24             else if ($_SESSION['usertype']=='1') { ?> src="addproduct.php" <?php }?> >
25         </frameset>
26     </frameset>
27     <body>
28
29     <p>此网页使用了框架，但您的浏览器不支持框架。</p>
30
31     </body>
32 </frameset>
33 </frameset>
34 </html>

```

top.php

```
1 <html>
2 <head>
3 </head>
4 <body>
5 <img src='images/logo.png'>
6 </body>
7 </html>
```

lefttree.php

```
<?php
// 设置页面缓存控制，禁止缓存
header("Cache-Control: no-cache, post-check=0, pre-check=0");

// 设置页面内容类型为 HTML，字符编码为 utf-8
header("Content-type:text/html;charset=utf-8");

// 启动会话，用于用户会话管理
session_start();

// 检查用户是否已登录（会话中 userid 是否为空），未登录则跳转到退出登录页面
if ($_SESSION['userid'] == '')
{
    // 通过 JavaScript 使顶层窗口跳转到 loginexit.php
    die('<script
language="javascript">top.location.href="loginexit.php"</script>');
}
?>

<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
<head id="Head1" runat="server">
    <title>无标题页</title>
    <!-- 以下三个 meta 标签均用于控制页面缓存，禁止缓存 -->
    <meta http-equiv="Pragma" content="no-cache">
    <meta http-equiv="Cache-Control" content="no-cache">
    <meta http-equiv="expires" content="0">
    <!-- 设置页面字符编码为 GBK -->
    <meta http-equiv="Content-Type" content="text/html; charset=GBK">
    <style>
        /* 全局样式设置，统一字体和大小 */
        BODY
        {
            font-family: ms shell dlg;
            font-size: 12px;
```

```
}  
  
DIV  
{  
    font-family: ms shell dlg;  
    font-size: 12px;  
}  
  
FONT  
{  
    font-family: ms shell dlg;  
    font-size: 12px;  
}  
  
/* 控制菜单项显示状态, off 为显示, on 为隐藏 */  
  
.off  
{  
    display: inline;  
}  
  
.on  
{  
    display: none;  
}  
  
/* 右键菜单样式 */  
  
.menus  
{  
    background-color: buttonface;  
    border-bottom: black 1px solid;  
    border-left: white 1px solid;  
    border-right: black 1px solid;  
    border-top: white 1px solid;  
    cursor: hand;  
    font-size: 9pt;  
    margin: 0px;  
    overflow: hidden;  
    padding-bottom: 1px;  
    padding-left: 6px;  
    padding-right: 6px;  
    padding-top: 1px;  
    text-align: left;  
    width: 90px;  
}  
  
/* 右键菜单定位样式 */  
  
#menutool  
{  
    left: 6px;
```

```

        position: absolute;
        width: 90px;
        z-index: 10; /* 确保菜单在顶层显示 */
    }
    /* 链接样式设置 */
    A
    {
        font-size: 9pt;
        text-decoration: none;
        text-transform: none;
        cursor: hand;
    }
    A:hover /* 鼠标悬停时链接样式 */
    {
        color: red;
        text-decoration: underline;
        cursor: pointer;
    }
    A:visited /* 已访问链接样式 */
    {
        cursor: hand;
    }
    .tab_border
    {
        border: 1px solid #000000;
    }
    #Form1
    {
        height: 231px;
    }
</style>

<script language="JavaScript">
<!--
// 打开修改密码的弹窗
function editUserInfo(){
    // 打开 Dialog_changepassword.aspx 页面，设置弹窗样式（无状态栏、菜单栏等，宽 240，
    高 140）
    window.open("Dialog_changepassword.aspx","", "status=0,menubar=0,location=0,
toolbar=0,width=240,height=140,scrollbars=1");
}

```

```

// 切换菜单展开/折叠状态
function swaping(myimgNum,secNum,folderimg){
    // 如果当前菜单是显示状态（off），则切换为隐藏状态（on），并更换图标
    if (secNum.className=="off")
    {
        secNum.className="on";
        myimgNum.src="images/tplus.gif"; // 切换为加号图标（表示可展开）
        folderimg.src="images/icon_folder.gif" // 切换为关闭的文件夹图标
    }
    else // 如果当前是隐藏状态，则切换为显示状态，并更换图标
    {
        secNum.className="off";
        myimgNum.src="images/tminus.gif"; // 切换为减号图标（表示可折叠）
        folderimg.src="images/icon_folderopen.gif" // 切换为打开的文件夹图标
    }
}

// 设置窗口默认状态文本为空
function statu()
{
    window.defaultStatus=' ';
}

// 每隔 10 毫秒执行一次 statu 函数（注释掉未启用）
//window.setInterval("statu()",10)

// 全部关闭菜单
function AllClose(){
    // 获取所有 div 标签
    tempColl = document.all.tags("div");

    // 遍历 div，将显示状态（off）的改为隐藏状态（on）
    for (i=0; i<tempColl.length; i++) {
        if (tempColl(i).className == "off") {
            tempColl(i).className = "on"
        }
    }

    // 获取所有 img 标签
    imgColl = document.all.tags("img");

    // 遍历图片，将菜单图标切换为折叠状态图标
    for (i=0; i<imgColl.length; i++) {

```

```

        if (imgColl(i).id != "") {
            if(imgColl(i).className == "havechild") {
                imgColl(i).src = "images/icon/icon_folder.gif" // 文件夹图标改为关闭状态
            }
            else{
                imgColl(i).src = "images/icon/tplus.gif" // 加减号图标改为加号（可展开）
            }
        }
    }
}

// 全部展开菜单
function AllOpen(){
    // 获取所有 div 标签
    tempColl = document.all.tags("div");
    // 遍历 div, 将隐藏状态 (on) 的改为显示状态 (off)
    for (i=0; i<tempColl.length; i++) {
        if (tempColl(i).className == "on") {
            tempColl(i).className = "off"
        }
    }
    // 获取所有 img 标签
    imgColl = document.all.tags("IMG");
    // 遍历图片, 将菜单图标切换为展开状态图标
    for (i=0; i<imgColl.length; i++) {

        if (imgColl(i).id != "") {
            if(imgColl(i).className == "havechild") {
                imgColl(i).src = "images/icon/icon_folderopen.gif" // 文件夹图标
改为打开状态
            }
            else{
                imgColl(i).src = "images/icon/tminus.gif" // 加减号图标改为减号（可
折叠）
            }
        }
    }
}

//-->

</script>

<script>

```

```

// 用于记录鼠标位置的变量
var Xpos = 1;
var Ypos = 1;
// 记录鼠标按下时的位置（相对于文档的滚动偏移+鼠标在可视区的位置）
function doDown()
{
    Xpos = document.body.scrollLeft+event.x;
    Ypos = document.body.scrollTop+event.y;
}

// 显示右键菜单，定位到鼠标按下的位置
function showmenu()
{
    menutool.style.left=Xpos;
    menutool.style.top=Ypos;
}
// 隐藏右键菜单
function hidemenu()
{
    menutool.style.display="none"
}
// 点击文档任意位置隐藏右键菜单
document.onclick = hidemenu;
// 鼠标按下时记录位置
document.onmousedown = doDown;
// 空函数，未使用
function load_page(link)
{
}

// 加载指定 URL 到父页面的 mailtreeframe 框架，并重置相关区域内容
function load_Href(url,sign)
{
    parent.mailtreeframe.location.href=url
    parent.messageframe.from.innerText=""
    parent.messageframe.subject.innerText=""
    parent.messageframe.view_source.innerText=""
    parent.messageframe.mailmsgArea.location.href="about:blank"
    parent.main_display.rows="160,*";
}

// 记录选中项的索引

```

```

var selectedIndex=0;

// 忽略自身错误
self.onError=null;

// 跳转到指定 URL，在父页面的 right 框架中显示
function go_Href(url)
{
    //if(!document.all)    return;

    if ((event.srcElement.className=="item"))
    {
        var srcIndex = event.srcElement.sourceIndex
        var nested = document.all[srcIndex+3]

        var srcIndex = event.srcElement.sourceIndex;
        var previous = document.all[selectedIndex];
        selectedIndex=srcIndex;
    }
    parent.right.location.href=url
}

</script>

</head>

<!-- body 样式及事件设置：
oncontextmenu: 右键点击时不显示默认菜单，显示自定义菜单
onselectstart: 禁止选中页面内容
ondragstart: 禁止拖动页面元素
设置边距 -->
<body class="lbody" oncontextmenu="self.event.returnValue=false;showmenu()"
onselectstart="self.event.returnValue=false"
ondragstart="self.event.returnValue=false" leftmargin="4" topmargin="8"
rightmargin="0">
<!-- 右键菜单容器，默认隐藏 -->
<div id="menutool" style="display: none">
    <table class="menus" cellspacing="0" cellpadding="0">
        <tbody>
            <tr>
                <td>
                    onmouseover="this.style.backgroundColor='darkblue';this.style.color='white';"
                    onclick="AllOpen();hidemenu()"

```


[illegible]

```

{ ?>

<!-- 购买商品菜单组标题 -->

<div id="a0" runat="server">

    <!-- 点击图标切换菜单展开/折叠状态 -->

     <!-- 加减号图
标 -->

         <!-- 文件夹图标 -->

        购买商品</div> <!-- 菜单组标题文字 -->

<!-- 购买商品菜单组内容（默认隐藏） -->

<div class="on" id="newfolderit48">

    <div id="t0" runat="server">

         <!-- 缩进图标 -->

         <!-- 连接线图标 -->

         <!-- 菜
单项图标 -->

        <a class="item" onclick="javascript:go_Href('selectproduct0.php')">选择
商品</a> <!-- 点击跳转到选择商品页面 -->

    </div>

    <div id="t1" runat="server">

        <a class="item" onclick="javascript:go_Href('orderlist.php')">订单查询
</a> <!-- 点击跳转到订单查询页面 -->

    </div>

    <div id="t2" runat="server">

        <a class="item"
onclick="javascript:go_Href('query_all/query_in_all.aspx')">投入查询</a> <!-- 点击跳
转到投入查询页面 -->

    </div>

    <div id="t3" runat="server">

        <a class="item"
onclick="javascript:go_Href('query_all/query_paifang.aspx')">液体排放查询</a> <!-- 点

```

击跳转到液体排放查询页面 -->

</div>

<div id="t4" runat="server">

<a class="item"

onclick="javascript:go_Href('query_all/query_update_all.aspx')">已修改查询 <!-- 点击跳转到已修改查询页面 -->

</div>

<div id="t5" runat="server">

<a class="item"

onclick="javascript:go_Href('query_all/query_over_out_all.aspx')">已结束产出查询 <!-- 点击跳转到已结束产出查询页面 -->

</div>

</div>

<!-- 工厂操作菜单组标题 -->

<div id="DIV86" runat="server">

 <!-- 加减号图标 -->

 <!-- 文件夹图标 -->

工厂操作</div> <!-- 菜单组标题文字 -->

<!-- 工厂操作菜单组内容（默认隐藏） -->

<div class="on" id="DIV87">

<div id="DIV91" runat="server">

新增工厂 <!-- 点击跳转到新增工厂页面 -->

</div>

<div id="DIV92" runat="server">

工厂列表 <!-- 点击跳转到工厂列表页面 -->

</div>

```

</div>

<!-- 注释掉的查询菜单组代码 -->
<!--
<DIV nowrap><IMG id="queryimg" style="CURSOR: hand"
onclick=swaping(queryimg,query,querying) alt="" src="images/t.gif" align=absMiddle
border=0>
        <IMG class=havechild id="querying" alt=""
src="images/icon_folderopen.gif" align=absMiddle border=0>
        <A class=item onclick="javascript:go_Href('inquiry.aspx')">查询</A>
</DIV>
-->

<!-- 修改密码菜单项 -->
<div nowrap>
         <!-- 加减号图标
-->
         <!-- 文件夹图标 -->
        <a class="item" onclick="javascript:editUserInfo()">修改密码</a> <!-- 点击打
开修改密码弹窗 -->
</div>

<?php } else if ($_SESSION['usertype'] == "1") // 用户类型为 1 时显示以下菜单
{?>
        <!-- 新增商品菜单组标题 -->
        <div id="a0" runat="server">
                 <!-- 加减号图
标 -->
                 <!-- 文件夹图标 -->
                新增商品</div> <!-- 菜单组标题文字 -->
        <!-- 新增商品菜单组内容（默认隐藏） -->
        <div class="on" id="newfolderit48">
                <div id="t0" runat="server">

```

```

         <!-- 缩进图标 -->

         <!-- 连接线图标 -->

         <!-- 菜
单项图标 -->

        <a class="item" onclick="javascript:go_Href('addproduct.php')">新增商品
</a> <!-- 点击跳转到新增商品页面 -->

    </div>

</div>

<?php } ?>

    <!-- 退出菜单项 -->

    <div>

         <!-- 连接线图标
-->

         <!-- 退
出图标 -->

        <a class="item" onclick=window.parent.location.href="loginexit.php" >退出</a>
<!-- 点击使父页面跳转到退出登录页面 -->

    </div>

</form>

</body>

```

Selectproduct0.php

```

1  <?php
2  // 启动会话，用于跟踪用户状态
3  session_start();
4  // 检查用户是否登录且用户类型为0（可能是普通用户），若未登录或类型不符则跳转至退出登录页面
5  if (($SESSION['userid'] == '') || ($SESSION['usertype'] <> '0'))
6  {
7      // 通过JavaScript跳转到loginexit.php页面（顶部窗口跳转）
8      die('<script language="javascript">top.location.href="loginexit.php"</script>');
9  }
10
11 // 设置缓存控制头，禁止缓存页面
12 header("Cache-Control: no-cache, post-check=0, pre-check=0");
13 // 设置页面内容类型为HTML，字符集为gb2312
14 header("Content-type:text/html;charset=gb2312");
15
16 // 连接MySQL数据库，服务器为localhost，用户名root，密码sql
17 $con = mysql_connect("localhost","root","sql");
18 // 选择数据库qing_zhou
19 mysql_select_db("qing_zhou", $con);
20 // 设置数据库查询字符集为gb2312，确保中文显示正常
21 mysql_query("set names gb2312 ");
22
23 // 获取POST提交的产品名称参数
24 $productname = $_POST['productname'];
25 // 构建SQL查询语句，根据产品名称模糊查询t_product表中的数据
26 $SQL = "select * from t_product where productname like '%". $productname ."%";
27
28 // 执行SQL查询，获取结果集
29 $RS0 = mysql_query($SQL, $con);
30 ?>
31
32 <HTML>
33 <HEAD>
34 <style type="text/css">
35 <!--
36 /* 设置字体样式：宋体，9pt */
37 font{font-family: "宋体";font-size:9pt}
38 /* 设置字体样式：宋体，14.3px */
39 .font1{font-family: "宋体";font-size:14.3px}
40 /* 设置表格单元格字体样式：宋体，9pt */
41 td{font-family: "宋体";font-size:9pt}

```

```

42  /* 去除超链接下划线 */
43  a{text-decoration:none}
44  -->
45  </style>
46
47  <script language=javascript>
48      // 处理添加订单的函数
49      function addorder()
50      {
51          // 定义正则表达式, 用于验证正整数
52          var r = /^\\+?[1-9][0-9]*$/;
53          // 获取所有名称为checkbox1的复选框元素 [选中的产品]
54          var checkboxes = document.getElementsByName("checkbox1");
55          // 获取所有名称为text1的文本框元素 [购买数量]
56          var texts = document.getElementsByName("text1");
57          // 定义数组用于存储选中的产品信息
58          var str = [];
59          // 遍历复选框
60          for(i=0;i<checkboxes.length;i++){
61              // 若复选框被选中
62              if(checkboxes[i].checked){
63                  // 将产品编码添加到数组
64                  str.push(checkboxes[i].value);
65                  // 验证购买数量是否为空或非正整数, 若不合法则提示并返回
66                  if (texts[i].value=="" || !(r.test(texts[i].value))) { alert('请输入正确数量'); return false; }
67                  // 将购买数量添加到数组
68                  str.push(texts[i].value);
69              }
70          }
71          // 将选中的产品信息数组转换为字符串, 存入隐藏域
72          document.getElementById("checkedproduct").value = str;
73          // 若未选择任何产品, 提示并返回
74          if( ""==str){
75              alert('请选择产品添加订单'); return false;
76          }else{
77              // 设置表单提交地址为addorder0.php并提交表单
78              document.form1.action = 'addorder0.php';
79              document.form1.submit();
80          }
81      }

```

```

82 </script>
83 </HEAD>
84 <TITLE>选择产品展示页面</TITLE>
85
86 <BODY>
87 <!-- 表单, 提交方式为POST -->
88 <form id=form1 name=form1 method=post>
89 <!-- 产品名称输入框和查询按钮, 点击查询按钮会提交表单, 触发产品查询 -->
90 产品名称<input type=text name=productname>
91 <input type=submit value=查询>
92 <!-- 收货地址输入框 -->
93 收货地址<input type=text name=address>
94 <!-- 生成订单按钮, 点击调用addorder()函数 -->
95 <input align=left type=button value=生成订单 onclick="return addorder();">
96 <!-- 隐藏域, 用于存储选中的产品编码和数量信息 -->
97 <input type=hidden id=checkedproduct name=checkedproduct>
98 </form>
99 <p></p>
100 <center>
101 <!-- 产品列表表格, 边框为0, 宽度100%, 单元格边距0, 间距1 -->
102 <table ID="SrhTable" border=0 width=100% cellpadding=0 cellspacing=1>
103 <tr>
104 <td></td>
105 <td>产品编码</td>
106 <td>产品名称</td>
107 <td>产品价格</td>
108 <td>产品库存</td>
109 <td>购买数量</td>
110 <td>产品图片</td>
111 </tr>
112
113 <?php
114 // 循环遍历查询结果集, 输出每条产品信息
115 while($RS = mysql_fetch_array($RS0))
116 {
117 ?>
118 <tr>
119 <!-- 复选框, 值为产品编码, 用于选择产品 -->
120 <td><input type=checkbox id="checkbox1" name="checkbox1"
121 | value="<?php echo strval($RS["productcode"])?>"></td>
122 <!-- 输出产品编码 -->
123 <td><?php echo strval($RS["productcode"])?></td>
124 <!-- 输出产品名称 -->
125 <td><?php echo strval($RS["productname"])?></td>
126 <!-- 输出产品价格 -->
127 <td><?php echo strval($RS["price"])?></td>
128 <!-- 输出产品库存数量 -->
129 <td><?php echo strval($RS["stocknumber"])?></td>
130 <!-- 购买数量输入框 -->
131 <td><input type=text id="text1" name="text1"></td>
132 <!-- 输出产品图片, 设置宽高为100px -->
133 <td><?php echo ""></td>
134 </tr>
135
136 <?php
137 }
138 // 关闭数据库连接
139 mysql_close($con);
140 // 释放结果集和连接变量
141 $RS=NULL;
142 $Con =NULL;
143 ?>
144
145 </table>
146
147 </html>

```